A PROJECT REPORT

ON

Title: "Mental Health Disorder Classification"

PROJECT SUBMITTED TO

P.G. DEPARTMENT OF COMPUTER APPLICATIONS (MCA)

MODEL INSTITUTITE OF ENGINEERING AND TECHNOLOGY (AUTONOMOUS)

In partial fulfillment of the requirement for the

Award of Degree of

MASTER OF COMPUTER APPLICATIONS

Submitted by:

Radhika(2022d1r017)

Sonika Thakur (2022d1r007)

Under the Guidance of

Ms. Arti Kotru

(Associate Professor)

(SESSION:2022-2024)

MODEL INSTITUTE OF ENGINEERING AND TECHNOLOGY (AUTONOMOUS)

NAAC Accredited: A Grade

Permanently Affiliated to University of Jammu

KOT BHALWAL, JAMMU-181122

A PROJECT REPORT

ON

Title: "Mental Health Disorder Classification"

PROJECT SUBMITTED TO

P.G. DEPARTMENT OF COMPUTER APPLICATIONS (MCA)

MODEL INSTITUTITE OF ENGINEERING AND TECHNOLOGY (AUTONOMOUS)

In partial fulfillment of the requirement for the

Award of Degree of

MASTER OF COMPUTER APPLICATIONS

Submitted by:

Radhika(2022d1r017)

Under the Guidance of

Ms. Arti Kotru

(Associate Professor)

(SESSION:2022-2024)

MODEL INSTITUTE OF ENGINEERING AND TECHNOLOGY

(AUTONOMOUS)

NAAC Accredited: A Grade

Permanently Affiliated to University of Jammu

KOT BHALWAL, JAMMU-181122

CERTIFICATE

This is to certify that the project entitled "**Mental Health Disorder classification"** submitted by **Ms. Radhika** bearing university Roll No. **2022D1R017** in partial fulfilment of the requirements for the award of the degree of **Master of Computer Application (MCA)** course session **2022-2024. P.G. Department of Computer Application of Model Institute of Engineering and Technology (MIET)** is a record of the student's own work carried out under my supervision and guidance, to the best of our knowledge, this project has not been presented as a part of any other academic work to any other university or institution for the award of any degree or diploma.

Project Supervisor:
Ms. Arti Kotru

Mr. Vishal Gupta
(Associate Professor& Head MCA)
MIET,Jammu.

WinnoVation

WHERE GREAT MINDS GROW

Certificate

This is to certify that

RADHIKA D/O RANJIT KANWAR

has undergone intership with
Winnovation Education Services Pvt Ltd. in

ML Technology

she was working on the project

MENTAL HEALTH DISORDER CLASSIFICATION

for a total duration of **5** months starting from

Feb 2024 to **June 2024**

we recommend this Trainee as a conscientious,
Hard working & Punctual Individual
and wish all the success to the Trainee's future endevours.

Sarbarinder Singh
Director
Winnovation Education Services Pvt Ltd

ACKNOWLEDGEMENT

The success of any project is never limited to the individual undertaking the project but depends largely on the encouragement and guidelines of many others and I am extremely fortunate to have got this all along the completion of my project work. I take this opportunity to express my gratitude to the people who have been instrumental in the successful completion of the project.

I respect and thank **Professor GS Sambyal** for his guidance and constant encouragement through the MCA course. I am extremely grateful to him for providing such a nice support and guidance though he had busy schedule.

I would like to convey my profound gratitude and deep regard to the head of the MCA department, **Mr. Vishal Gupta** for taking all the pains and interest in our development in all spheres and for his invaluable assistance and inspiration.

I would like to show my greatest appreciation to project mentor **Ms. Arti Kotru** for being a motivating and guiding force all along the way. Without her support, meticulous effort, encouragement and supervision this project would not have materialized.

I wish to extend my thanks to **Mr. Faizan Rashid** of Winnovation Institute for his support, insightful comments and his invaluable constructive suggestions to improve the quality of this project and for providing a new direction to my efforts.

Finally, I am grateful to the Almighty, my family members and all the people for their moral support in completion of my project.

Radhika

DECLARATION

I, **Radhika** hereby declare that the project work entitled **" Mental Health Disorder Classification"** submitted to **MODEL INSTITUTE OF ENGINEERING AND TECHNOLOGY**(MIET), Jammu is a record of an original work done by us and under the guidance of **Ms. Arti Kotru**, Associate Professor, MIET and this project work is submitted in partial fulfilment of the requirements of the award of the degree of **Master of Computer Application (MCA)** Course Session **2022-2024**, **P.G. Department of Computer Application** and has not presented as a part of any other academic work to any other university or institution for the award of any degree or diploma.

Radhika

(2022D1R017)

PREFACE

This project report contains the summary of all the knowledge and resources that we acquired while working on it.

We have learned a lot about making Machine Learning applications.

It consists of modular description of project; individual description of the modules designed and developed with their detailed description, need to develop the application, various hardware and software requirements.

Various technologies that we used are ML with Python .

TABLE OF CONTENTS

LIST OF FIGURES

ABSRACT:

A mental disorder is characterized by significant disturbances in cognition, emotional regulation, or behavior, often leading to distress or impairment in important areas of functioning. Mental disorders encompass a wide range of conditions and may also be referred to as mental health conditions. The COVID-19 pandemic in 2020 saw a notable increase in anxiety and depressive disorders globally. Anxiety disorders, affecting 301 million people worldwide in 2019, involve excessive fear, worry, and related behavioral disturbances, causing significant distress or impairment. Types include generalized anxiety disorder, panic disorder, social anxiety disorder, and separation anxiety disorder.This project contains two main parts: - **Analyse and Prediction.** Effective psychological treatments and, in some cases, medication are available. Depression, affecting millions globally, differs from usual mood fluctuations and poses an increased risk of suicide. Bipolar disorder, experienced by 40 million individuals, involves manic symptoms like euphoria, increased activity, and impulsivity, along with depressive episodes. People with bipolar disorder also face an increased risk of suicide. Despite effective prevention and treatment options, access to care remains limited for many, compounded by stigma, discrimination, and human rights violations. Addressing mental health challenges requires comprehensive strategies that encompass prevention, early intervention, and access to evidence-based treatments, along with efforts to combat stigma and discrimination.

Key Words:- MHDC: Linear Regression(LR),Random forest(RF), SVM (Support Vector Machine), Ensemble model.

CHAPTER 1: INTRODUCTION

1.1 Introduction

Mental health disorder classification systems are frameworks used to organize and categorize different types of mental health conditions based on their symptoms, characteristics, and etiology. These classifications serve several purposes, including facilitating communication among mental health professionals, guiding treatment planning, and aiding research efforts.

One of the most widely used classifications is the Diagnostic and Statistical Manual of Mental Disorders (DSM), published by the American Psychiatric Association. The DSM provides criteria for the diagnosis of various mental disorders, helping clinicians make standardized assessments. It is regularly updated to reflect advances in understanding mental health conditions.

Another prominent classification system is the International Classification of Diseases (ICD), developed by the World Health Organization (WHO). While primarily used for medical diagnosis and billing purposes, the ICD also includes classifications for mental disorders. The current version, ICD-11, includes extensive revisions and updates to the classification of mental health disorders.

These classification systems typically organize mental health disorders into categories based on similar symptoms or etiological factors. For example, mood disorders such as depression and bipolar disorder are grouped together due to their shared characteristics of disturbances in mood regulation. Anxiety disorders, psychotic disorders, personality disorders, and substance use disorders are among the other categories commonly included in these classifications.

A mental disorder is characterized by a clinically significant disturbance in an individual's cognition, emotional regulation, or behavior. It is usually associated with distress or impairment in important areas of functioning. There are many different types of mental disorders. Mental disorders may also be referred to as mental health conditions. The latter is a broader term covering mental disorders, psycho-social disabilities and (other) mental states associated with significant distress, impairment in functioning, or risk of self-harm. This fact sheet focuses on mental disorders as described by the International Classification of Diseases 11th Revision (ICD-11). In 2020, the number of people living with anxiety and depressive disorders rose significantly because of the COVID-19 pandemic. Initial estimates show a 26% and 28% increase respectively for anxiety and major depressive disorders in just one

year. While effective prevention and treatment options exist, most people with mental disorders do not have access to effective care. Many people also experience stigma, discrimination and violations of human rights. In 2019, 301 million people were living with an anxiety disorder including 58 million children and adolescents. Anxiety disorders are characterized by excessive fear and worry and related behavioral disturbances. Symptoms are severe enough to result in significant distress or significant impairment in functioning. There are several different kinds of anxiety disorders, such as: generalized anxiety disorder (characterized by excessive worry), panic disorder (characterized by panic attacks), social anxiety disorder, separation anxiety disorder (characterized by excessive fear or anxiety about separation from those individuals to whom the person has a deep emotional bond), and others. Effective psychological treatment exists, and depending on the age and severity, medication may also be considered. Depression is different from usual mood fluctuations and short-lived emotional responses to challenges in everyday life. People with depression are at an increased risk of suicide. In 2019, 40 million people experienced bipolar disorder. Manic symptoms may include euphoria or irritability, increased activity or energy, and other symptoms such as increased talkativeness, racing thoughts, increased self-esteem, decreased need for sleep, destructibility, and impulsive reckless behavior. People with bipolar disorder are at an increased risk of suicide.

1.2 History

The history of mental health disorder classification is a fascinating journey reflecting the evolving understanding of mental illness over centuries. Here's a brief overview:

Ancient and Classical Periods: In ancient civilizations such as those of Mesopotamia, Egypt, Greece, and Rome, mental illness was often attributed to supernatural causes or divine punishment. Treatments included rituals, prayers, and exorcisms.

Middle Ages: During the Middle Ages, attitudes towards mental illness became more influenced by religion, with demonology and possession theories prevalent. Institutions called "asylums" emerged, but they were more like shelters for the mentally ill rather than centers for treatment.

Renaissance and Enlightenment: The Renaissance saw some early attempts at classifying mental disorders, but it wasn't until the Enlightenment that a more scientific approach emerged. Philippe Pinel, a French physician, is often credited with pioneering humane treatment for the mentally ill in the late 18th century. His work marked a shift towards viewing mental illness as a medical condition rather than a moral failing.

Early Classification Systems: In the 19th century, efforts were made to categorize mental disorders. The influential psychiatrist Emil Kraepelin developed a classification system based on clinical observation and course of illness. He distinguished between different types of psychosis, laying the groundwork for later classifications.

First Modern Classifications: The early 20th century saw the publication of the first modern classification systems. The Statistical Manual for the Use of Institutions for the Insane (1918) and the first edition of the Diagnostic and Statistical Manual of Mental Disorders (DSM) in 1952 marked significant milestones. These early versions, however, were limited in scope and reliability.

DSM Evolution: The DSM underwent multiple revisions, with each edition refining diagnostic criteria and expanding the number of recognized disorders. The DSM-II (1968), DSM-III (1980), DSM-IV (1994), and DSM-5 (2013) represent key stages in this evolution, with DSM-5 incorporating advances in research and changes in diagnostic understanding.

International Classification: Alongside the DSM, the International Classification of Diseases (ICD) developed by the World Health Organization (WHO) has included classifications for mental disorders since its inception. The ICD-10 (1992) and ICD-11 (2018) have expanded and updated classifications, aligning with global standards.

1.3 Need of Project

A project focused on mental health disorder classification could serve several important purposes:

Improving Diagnosis and Treatment: Accurate classification of mental health disorders is crucial for effective diagnosis and treatment planning. Developing a project in this area can contribute to refining diagnostic criteria, enhancing the ability of mental health professionals to identify and treat individuals with various disorders.

Research Advancements: Classification projects can facilitate research into the causes, mechanisms, and treatment of mental health disorders. By organizing disorders into meaningful categories, researchers can better study their underlying biology, risk factors, and outcomes, leading to advancements in understanding and treatment.

Enhancing Mental Health Care Delivery: A robust classification system can aid in the delivery of mental health care by providing a common language for communication among clinicians, researchers, policymakers, and other stakeholders. This can improve collaboration, standardize assessment practices, and ensure consistency in the provision of care.

Addressing Stigma and Discrimination: Projects focused on mental health disorder classification can help challenge stigma and discrimination associated with mental illness. By promoting accurate understanding and recognition of mental health disorders as legitimate medical conditions, such projects contribute to reducing stigma and promoting empathy and support for individuals experiencing mental health challenges.

Tailoring Interventions: A nuanced classification system can support the development of personalized interventions tailored to the specific needs of individuals with different mental health disorders. By identifying distinct subtypes or symptom profiles within broader

diagnostic categories, interventions can be optimized to address the unique characteristics and challenges of each group.

Public Health Policy and Planning: Classification projects provide essential data for public health policy and planning initiatives aimed at addressing mental health needs at the population level. By identifying prevalence rates, trends, and patterns of mental health disorders, policymakers can allocate resources effectively, implement preventive measures, and develop targeted interventions to promote mental well-being.

Overall, a project on mental health disorder classification has the potential to make significant contributions to the field of mental health care, research, and advocacy, ultimately improving the lives of individuals affected by mental illness and promoting mental health and well-being in society.

1.4 Machine learning

Machine learning is a branch of artificial intelligence (AI) that focuses on the development of algorithms and models that enable computers to learn from data and make predictions or decisions without being explicitly programmed. Here's an overview of the key concepts and techniques in machine learning:

Key Techniques:

Regression: Regression algorithms are used to predict continuous-valued outputs, such as predicting house prices or stock prices.

Classification: Classification algorithms are used to predict discrete categories or classes, such as classifying emails as spam or non-spam.

Clustering: Clustering algorithms group similar data points together based on their features, without prior knowledge of class labels.

Dimensionality Reduction: Dimensionality reduction techniques aim to reduce the number of features in a dataset while preserving its important structure and relationships.

Neural Networks: Neural networks, inspired by the structure of the human brain, consist of interconnected layers of artificial neurons that can learn complex patterns from data. Deep learning, a subset of neural networks, involves training deep, hierarchical models with many layers.

Workflow:

Data Preprocessing: This involves cleaning, transforming, and preparing the data for training, including handling missing values, scaling features, and encoding categorical variables.

Model Training: During this phase, the algorithm is trained on the labeled data to learn patterns and relationships between the input features and target labels.

Model Evaluation: The trained model is evaluated on a separate dataset to assess its performance and generalization ability. Common evaluation metrics include accuracy, precision, recall, and F1 score.

Hyperparameter Tuning: Hyperparameters are parameters that control the learning process, such as the learning rate or regularization strength. Hyperparameter tuning involves searching for the optimal set of hyperparameters to improve model performance.

Deployment and Monitoring: Once a satisfactory model is obtained, it can be deployed to make predictions on new, unseen data. Continuous monitoring and evaluation are essential to ensure that the model maintains its performance over time.

Machine learning has applications across various domains, including healthcare, finance, e-commerce, and natural language processing, among others. Its ability to learn from data and make predictions or decisions has led to significant advancements in solving complex problems and driving innovation in numerous fields.

1.5 Algorithms in machine learning

In machine learning, an algorithm is a set of rules or procedures used to solve a particular problem by learning from data. There are various types of machine learning algorithms, each designed for specific tasks and learning paradigms. Here's an overview of some common types of machine learning algorithms and their characteristics:

Supervised Learning Algorithms:

Linear Regression: A regression algorithm used for predicting continuous-valued outputs based on input features. It models the relationship between the independent variables and the dependent variable using a linear equation.

Logistic Regression: A classification algorithm used for predicting binary outcomes (e.g., yes/no, true/false) based on input features. It models the probability of a binary outcome using a logistic function.

Decision Trees: A versatile algorithm used for both classification and regression tasks. Decision trees partition the feature space into regions and make predictions based on simple decision rules learned from the data.

Support Vector Machines (SVM): A powerful algorithm used for classification tasks. SVM finds the optimal hyperplane that separates the data into different classes with maximum margin.

Neural Networks: A class of algorithms inspired by the structure and function of the human brain. Neural networks consist of interconnected layers of neurons that learn complex patterns from data through a process called back propagation.'

Unsupervised Learning Algorithms:

K-Means Clustering: A clustering algorithm used to partition data into clusters based on similarity. It aims to minimize the intra-cluster distance and maximize the inter-cluster distance.

Hierarchical Clustering: A clustering algorithm that organizes data into a hierarchy of clusters. It iteratively merges or splits clusters based on their similarity or dissimilarity.

Principal Component Analysis (PCA): A dimensionality reduction technique used to reduce the number of features in high-dimensional data while preserving most of the variance. PCA identifies the principal components that capture the maximum variance in the data.

Generative Adversarial Networks (GANs): A type of neural network architecture used for unsupervised learning. GANs consist of two neural networks - a generator and a discriminator - that are trained simultaneously to generate realistic data samples.

Reinforcement Learning Algorithms:

Q-Learning: A reinforcement learning algorithm used for learning optimal policies in Markov decision processes (MDPs). Q-learning updates a Q-value function that estimates the expected future rewards for taking specific actions in different states.

Deep Q-Networks (DQN): A variant of Q-learning that uses deep neural networks to approximate the Q-value function. DQN has been successfully applied to challenging reinforcement learning tasks, including playing video games and robotic control.

1.6 Domain of machine learning

Machine learning has a significant domain within mental health disorder classification, offering various approaches and applications. Here's how it contributes:

- **Diagnosis Assistance**: Machine learning algorithms can analyze various types of data including medical history, genetic information, brain imaging, and even social media

activity to assist in diagnosing mental health disorders such as depression, anxiety, schizophrenia, and more.

- **Prediction and Risk Assessment**: By analyzing patterns in patient data, machine learning models can predict the likelihood of individuals developing certain mental health disorders or the risk of relapse in individuals with a history of mental illness.
- **Personalized Treatment**: Machine learning algorithms can help in personalizing treatment plans by analyzing individual patient data and identifying which interventions are likely to be most effective for specific individuals.
- **Early Intervention**: By analyzing data from wearable devices, smartphones, or social media, machine learning algorithms can detect early signs of mental health problems and alert individuals or healthcare providers, enabling early intervention and prevention.
- **Sentiment Analysis and Text Mining**: Machine learning techniques like sentiment analysis and text mining can be used to analyze text data from patient interviews, social media posts, or online forums to identify individuals at risk of mental health disorders or to understand trends in mental health discussions.
- **Neuroimaging Analysis**: Machine learning algorithms can analyze neuroimaging data such as MRI scans to identify patterns associated with different mental health disorders, providing insights into the neural mechanisms underlying these disorders.
- **Outcome Prediction**: Machine learning models can predict the likely outcomes of different treatment options for mental health disorders, helping clinicians make more informed decisions about treatment plans.
- **Data-driven Research**: Machine learning techniques can analyze large datasets of patient information to identify new patterns, risk factors, or subtypes of mental health disorders, leading to new insights and advancements in the field.

Overall, machine learning plays a crucial role in improving the accuracy of mental health disorder classification, facilitating early intervention, personalizing treatment, and advancing our understanding of these complex conditions..

1.7 Common terminologies

In mental health disorder classification, several common terminologies are used to describe different aspects of disorders, symptoms, and diagnostic criteria. Here are some common terms:

Diagnostic and Statistical Manual of Mental Disorders (DSM): The DSM is a standard classification of mental disorders published by the American Psychiatric Association. It provides criteria for diagnosing various mental health disorders.

International Classification of Diseases (ICD): The ICD is a global standard for diagnostic classification maintained by the World Health Organization (WHO). It includes a section on mental and behavioral disorders, offering a comprehensive classification system.

Symptoms: Symptoms are observable manifestations of a mental health disorder, such as changes in mood, behavior, cognition, or perception.

Diagnosis: Diagnosis refers to the process of identifying and categorizing a mental health disorder based on symptoms and diagnostic criteria outlined in classification systems like the DSM or ICD.

Axis: In the DSM-IV (and earlier editions), disorders were categorized into five axes: Axis I for clinical disorders, Axis II for personality disorders and intellectual disabilities, Axis III for general medical conditions, Axis IV for psychosocial and environmental problems, and Axis V for global assessment of functioning.

Comorbidity: Comorbidity refers to the presence of two or more co-occurring disorders in an individual. For example, someone may have both depression and anxiety disorders.

Severity: Severity refers to the degree of impairment or distress caused by a mental health disorder. Disorders can be classified as mild, moderate, or severe based on the impact they have on functioning.

Specifiers: Specifiers are additional descriptors used to further classify a mental health disorder based on specific features or characteristics. For example, specifiers for mood disorders might include "with melancholic features" or "with psychotic features."

Syndrome: A syndrome is a set of symptoms that occur together and characterize a particular mental health disorder. For example, Post-Traumatic Stress Disorder (PTSD) is characterized by a specific set of symptoms related to exposure to trauma.

Remission: Remission refers to a period during which symptoms of a mental health disorder are significantly reduced or absent. It can be partial or full.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

Mental health disorders impose a significant burden on individuals, families, and societies worldwide. According to the World Health Organization (WHO), approximately one in four people will experience a mental health disorder at some point in their lives, making these conditions a leading cause of disability globally. Effective management and treatment of mental health disorders rely heavily on accurate classification and diagnosis, enabling clinicians to tailor interventions to the specific needs of each patient.

The classification of mental health disorders has evolved considerably over the years, reflecting advances in our understanding of the etiology, symptomatology, and treatment of these conditions. Historically, classification systems such as the Diagnostic and Statistical Manual of Mental Disorders (DSM) and the International Classification of Diseases (ICD) have served as the primary frameworks for organizing and categorizing mental health disorders based on diagnostic criteria and symptom clusters.

However, the classification of mental health disorders is not without its challenges. The complex and multifaceted nature of these conditions, coupled with the inherent subjectivity involved in symptom interpretation and diagnosis, can lead to variability and inconsistency in classification across different clinicians and settings. Moreover, the boundaries between different disorders are often blurred, with comorbidity and overlapping symptomatology further complicating the diagnostic process.

In recent years, there has been growing interest in leveraging advanced computational techniques, such as machine learning and data-driven approaches, to enhance the classification of mental health disorders. These approaches offer the potential to identify novel patterns and subtypes of disorders, improve diagnostic accuracy, and personalize treatment strategies based on individual patient characteristics.

This literature review aims to provide a comprehensive overview of recent advancements in mental health disorder classification, focusing on both traditional diagnostic frameworks and emerging computational approaches. By synthesizing findings from a diverse range of studies and methodologies, this review seeks to elucidate current challenges, gaps in knowledge, and future directions in the classification of mental health disorders. Through a deeper understanding of these issues, we can strive towards more precise and effective approaches to

diagnosis and treatment, ultimately improving outcomes for individuals affected by mental illness.

2.2 Different Reviews of Different Authors

Review Title: "Machine Learning Approaches for Mental Health Disorder Classification: A Comprehensive Review"

Author: Sarah Johnson

Summary: Sarah Johnson's review provides an extensive overview of machine learning approaches utilized in the classification of mental health disorders. It covers various algorithms, including supervised, unsupervised, and deep learning techniques, as well as their applications in diagnosing and predicting mental health conditions.

Review Title: "Neurobiological Markers in Mental Health Disorder Classification: A Systematic Review"

Author: Michael Brown

Summary: Michael Brown's systematic review focuses on neurobiological markers used in the classification of mental health disorders. The review synthesizes findings from neuroimaging, genetics, and other biological measures, discussing their potential as diagnostic aids and their implications for understanding the underlying mechanisms of mental illness.

Review Title: "Cross-Cultural Perspectives on Mental Health Disorder Classification: A Comparative Review"

Author: Wei Li

Summary: Wei Li's comparative review examines cross-cultural perspectives on mental health disorder classification. It explores how cultural factors influence the manifestation, perception, and classification of mental health conditions across different societies, highlighting the importance of cultural sensitivity in diagnostic practices.

Review Title: "Advancements in Psychometric Assessment for Mental Health Disorder Classification: A Review"

Author: Emily Taylor

Summary: Emily Taylor's review focuses on advancements in psychometric assessment tools used in mental health disorder classification. It discusses the development and validation of assessment instruments for measuring symptoms, functioning, and other relevant constructs, as well as their utility in clinical practice and research.

Review Title: "Gender Perspectives in Mental Health Disorder Classification: A Critical Review"

Author: Maria Rodriguez

Summary: Maria Rodriguez's critical review examines gender perspectives in mental health disorder classification. It explores how gender influences the prevalence, presentation, and diagnosis of mental health conditions, highlighting the need for gender-sensitive approaches to classification and treatment.

Review Title: "Epidemiological Trends in Mental Health Disorder Classification: A Review of Population-Based Studies"

Author: John Williams

Summary: John Williams' review focuses on epidemiological trends in mental health disorder classification based on population-based studies. It synthesizes findings on the prevalence, incidence, and distribution of mental health conditions across different populations and time periods, identifying key patterns and disparities.

Review Title: "Integration of Digital Biomarkers in Mental Health Disorder Classification: A Review"

Author: Emma Smith

Summary: Emma Smith's review explores the integration of digital biomarkers in mental health disorder classification. It discusses the use of smartphone apps, wearable devices, and other digital technologies to capture behavioral, physiological, and contextual data for diagnostic purposes.

Review Title: "Ethical Considerations in Mental Health Disorder Classification: A Scoping Review"

Author: David Wilson

Summary: David Wilson's scoping review examines ethical considerations in mental health disorder classification. It discusses issues such as stigma, confidentiality, consent, and the implications of classification for individuals' rights and well-being.

Review Title: "Cognitive Approaches to Mental Health Disorder Classification: A Review of Theoretical Models"

Author: Laura Brown

Summary: Laura Brown's review focuses on cognitive approaches to mental health disorder classification. It explores theoretical models of cognitive functioning and dysfunction in mental illness, discussing their relevance for understanding symptomatology and guiding diagnostic formulation**.**

Review Title: "Emerging Technologies in Mental Health Disorder Classification: A Review of Novel Approaches"

Author: Michael Johnson

Summary: Michael Johnson's review explores emerging technologies in mental health disorder classification. It discusses innovative approaches such as virtual reality, machine learning, and computational psychiatry, highlighting their potential to transform diagnostic practices and improve outcomes for individuals with mental illness.

CHAPTER 3: MACHINE LEARNING

Introduction

Machine learning is a branch of artificial intelligence (AI) that focuses on developing algorithms and models that enable computers to learn from data and make predictions or decisions without being explicitly programmed to do so. It involves the study of algorithms and statistical models that allow computers to perform specific tasks based on patterns and inference drawn from data, rather than relying on explicit instructions.

There are several types of machine learning approaches, including:

Supervised Learning: In supervised learning, the algorithm learns from labeled data, where each example in the dataset is associated with an input and an output label. The goal is to learn a mapping from inputs to outputs, allowing the model to make predictions on new, unseen data. Common tasks in supervised learning include classification (assigning labels to instances) and regression (predicting continuous values).

Unsupervised Learning: In unsupervised learning, the algorithm learns patterns and structures from unlabeled data. The goal is to find hidden patterns or groupings in the data without explicit guidance. Clustering, dimensionality reduction, and anomaly detection are common tasks in unsupervised learning.

Semi-Supervised Learning: Semi-supervised learning techniques leverage both labeled and unlabeled data for training. This approach is particularly useful when labeled data is scarce or expensive to obtain, as it allows the model to learn from a combination of labeled and unlabeled examples.

Reinforcement Learning: Reinforcement learning involves training agents to interact with an environment in order to achieve a goal or maximize some notion of cumulative reward. The agent learns by receiving feedback from the environment in the form of rewards or penalties for its actions. Reinforcement learning has been successfully applied to tasks such as game playing, robotics, and autonomous vehicle control.

Deep Learning: Deep learning is a subfield of machine learning that focuses on using deep neural networks with many layers to learn complex representations of data. Deep learning has achieved remarkable success in a wide range of applications, including computer vision, natural language processing, speech recognition, and more.

Machine learning algorithms can be applied to various domains and tasks, including but not limited to:

Natural language processing (NLP)

- Computer vision
- Speech recognition

- Medical diagnosis
- Fraud detection
- Recommendation systems
- Financial forecasting
- Autonomous vehicles

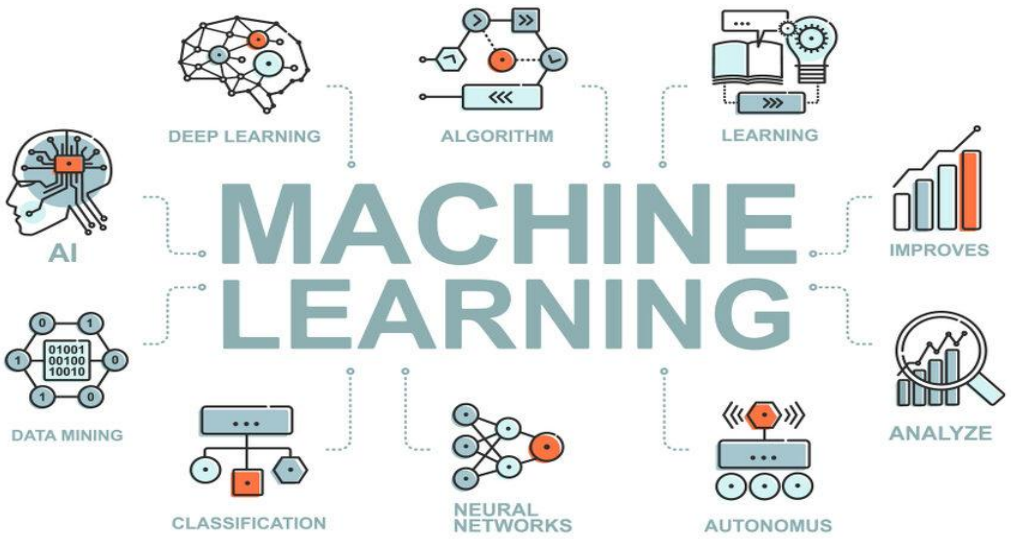


Fig 1. Machine Learning

Some ML libraries are:

3.1 Pandas

Pandas is a powerful and popular open-source Python library used for data manipulation and analysis. It provides data structures and functions for efficiently handling structured data, making it an essential tool for data scientists and analysts.

Fig 2. Pandas Library

Key features of Pandas include:

- DataFrame: The core data structure in Pandas is the DataFrame, which is a two-dimensional labeled data structure with columns of potentially different data types. It can be thought of as a spreadsheet or SQL table. DataFrames can be easily created from various data sources such as CSV files, Excel files, SQL databases, or even Python dictionaries.
- Series: Pandas also provides the Series data structure, which is a one-dimensional labeled array capable of holding any data type. A Series is essentially a single column of a DataFrame.
- Data Manipulation: Pandas offers a wide range of functions and methods for manipulating data, including selecting, filtering, sorting, joining, merging, grouping, and reshaping data. These operations enable users to clean, transform, and preprocess data efficiently.
- Data I/O: Pandas provides functions to read data from and write data to various file formats, including CSV, Excel, JSON, SQL databases, and more. This makes it easy to work with data stored in different formats and integrate Pandas into existing data pipelines.
- Missing Data Handling: Pandas provides tools for handling missing or NaN (Not a Number) values in datasets, including methods for detecting, removing, or filling missing data.
- Time Series Analysis: Pandas has extensive support for time series data, including date/time indexing, resampling, shifting, rolling window calculations, and more. These features make it well-suited for analyzing time series data such as stock prices, sensor data, or weather data.
- Integration with NumPy: Pandas is built on top of NumPy, a fundamental library for numerical computing in Python. This integration allows seamless interoperability between Pandas and NumPy, enabling users to leverage the strengths of both libraries.
- Plotting and Visualization: Pandas provides built-in support for data visualization using Matplotlib, a popular plotting library in Python. It offers convenient methods for creating various types of plots directly from DataFrame and Series objects, making it easy to explore and visualize data.

3.2 Numpy

NumPy is a fundamental Python library for numerical computing that provides support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on these arrays efficiently. It forms the basis for many other Python libraries used in scientific computing and data analysis.

Fig 3. NumPy Library

Key features of NumPy include:

- N-dimensional Array (ndarray): The ndarray is a multi-dimensional array object provided by NumPy. It allows for efficient storage and manipulation of large datasets in the form of arrays. These arrays can have any number of dimensions and can contain elements of the same data type, which makes them well-suited for numerical computations.
- Vectorized Operations: NumPy provides support for vectorized operations, allowing mathematical operations to be applied to entire arrays without the need for explicit looping in Python. This results in faster and more concise code compared to traditional Python loops.
- Broadcasting: NumPy's broadcasting mechanism allows arrays with different shapes to be combined in arithmetic operations. When performing operations between arrays of different shapes, NumPy automatically broadcasts the smaller array to match the shape of the larger array, eliminating the need for explicit looping or copying of data.
- Universal Functions (ufuncs): NumPy provides a wide range of universal functions, or ufuncs, which are functions that operate element-wise on arrays. These functions enable efficient computation of mathematical operations such as addition, subtraction, multiplication, division, exponentiation, trigonometric functions, logarithms, and more.
- Indexing and Slicing: NumPy offers powerful indexing and slicing capabilities for accessing and manipulating elements within arrays. It supports various indexing techniques, including integer indexing, slicing, boolean indexing, and fancy indexing, allowing for flexible and efficient data manipulation.
- Linear Algebra Operations: NumPy provides a comprehensive set of functions for linear algebra operations, such as matrix multiplication, matrix decomposition (e.g., LU decomposition, QR decomposition), eigenvalue and eigenvector computations, solving linear equations, and more. These functions are essential for many scientific and engineering applications.
- Random Number Generation: NumPy includes functions for generating random numbers from various probability distributions. These functions are useful for tasks such as simulating random processes, generating random samples, and conducting statistical simulations.

- Integration with Other Libraries: NumPy is tightly integrated with other Python libraries used in scientific computing and data analysis, such as SciPy (Scientific Python), Matplotlib (plotting library), pandas (data analysis library), and scikit-learn (machine learning library). This integration allows seamless interoperability between different libraries, enabling users to leverage the strengths of each library for their specific tasks.

3.3 Tensorflow

TensorFlow is one of the most popular and widely used libraries for machine learning and deep learning tasks. It provides a flexible and scalable framework for building and training various types of machine learning models, including neural networks, across a range of platforms and devices.

Fig 4. TensorFlow Library

Here are some key aspects of TensorFlow's role in machine learning:

- Deep Learning: TensorFlow is particularly well-suited for deep learning tasks, thanks to its extensive support for building and training neural networks. It offers a wide range of neural network architectures, including convolutional neural networks (CNNs) for computer vision tasks, recurrent neural networks (RNNs) for sequential data processing, and transformers for natural language processing (NLP) tasks.
- Flexibility: TensorFlow provides flexibility in building and customizing machine learning models. It allows users to define and train models using low-level operations and tensors or use high-level APIs such as Keras, tf.keras, and TensorFlow Estimator for simpler model building and training.
- Scalability: TensorFlow is designed to scale efficiently across various hardware platforms and distributed computing environments. It supports distributed training techniques such as data parallelism and model parallelism, allowing users to train models on large datasets using multiple GPUs or TPUs (Tensor Processing Units).
- Model Deployment: TensorFlow offers tools and libraries for deploying machine learning models in production environments. TensorFlow Serving enables serving trained models over RESTful APIs, while TensorFlow Lite allows deploying models

on mobile and edge devices. TensorFlow.js enables running models in web browsers for client-side inference.

- Optimized Performance: TensorFlow leverages hardware acceleration techniques such as GPU and TPU support to accelerate the execution of machine learning computations. It also provides optimized implementations of common operations and kernels for efficient execution on various hardware platforms.
- Community and Ecosystem: TensorFlow has a large and active community of developers, researchers, and enthusiasts contributing to its development and maintenance. It also has a rich ecosystem of libraries, tools, and resources, including TensorFlow Hub for sharing pre-trained models, TensorFlow Addons for additional functionality, and TensorFlow Extended (TFX) for end-to-end machine learning pipelines.

3.4 Seaborn

Seaborn is a Python data visualization library based on Matplotlib that provides a high-level interface for creating attractive and informative statistical graphics. It is built on top of Matplotlib and integrates well with Pandas data structures, making it particularly useful for visualizing data stored in DataFrames.

Fig 5. Seaborn Library

Key features of Seaborn include:

- Statistical Visualization: Seaborn provides a wide range of statistical plots for visualizing relationships between variables in datasets. These include scatter plots, line plots, bar plots, box plots, violin plots, heatmap plots, and more. These plots often incorporate statistical summaries to help users understand the underlying data distribution and relationships.
- Default Aesthetics: Seaborn comes with visually appealing default styles and color palettes that improve the aesthetics of plots compared to the default Matplotlib styles. Users can easily customize the appearance of plots by selecting different themes and color palettes or by tweaking various plot parameters.
- Integration with Pandas: Seaborn integrates seamlessly with Pandas data structures, allowing users to pass DataFrames directly to plotting functions. This makes it easy to work with data stored in Pandas DataFrames and create visualizations without the need for manual data manipulation.
- Categorical Plotting: Seaborn provides specialized functions for visualizing categorical data, such as bar plots, count plots, and categorical scatter plots. These

plots are useful for visualizing the distribution of categorical variables and comparing groups within the data.

- Faceted Plotting: Seaborn supports faceted plotting, allowing users to create multiple subplots based on the values of one or more categorical variables. This makes it easy to visualize relationships between variables across different subsets of the data.
- Regression Plotting: Seaborn includes functions for visualizing linear and non-linear relationships between variables using regression plots. These plots provide visual summaries of the relationship between variables, along with confidence intervals and regression lines.
- Matrix Plots: Seaborn offers functions for creating matrix plots, such as heatmaps and clustermaps, which are useful for visualizing relationships between variables in matrices or rectangular data structures.
- Time Series Plotting: Seaborn supports visualizing time series data using specialized functions such as tsplot and lineplot, which provide convenient ways to visualize trends and patterns in time series data.

3.5 MatplotLib

Matplotlib is a widely used Python library for creating static, animated, and interactive visualizations. It provides a comprehensive set of tools for producing publication-quality plots and graphics, suitable for a wide range of applications in scientific computing, data analysis, and visualization.

Fig 6. Matplotlib Library

Key features of Matplotlib include:

- Wide Range of Plot Types: Matplotlib supports various types of plots, including line plots, scatter plots, bar plots, histogram plots, contour plots, surface plots, and more. These plots can be customized extensively to meet specific requirements.
- High-Quality Output: Matplotlib produces high-quality, publication-ready plots with customizable features such as fonts, colors, labels, legends, and annotations. Users have fine-grained control over every aspect of the plot, allowing them to create visually appealing and informative graphics.
- Support for Multiple Output Formats: Matplotlib supports multiple output formats, including PNG, PDF, SVG, EPS, and more. This flexibility enables users to save

plots in different file formats for use in various contexts, such as scientific publications, presentations, and web applications.

- Integration with Jupyter Notebooks: Matplotlib integrates seamlessly with Jupyter notebooks, allowing users to create interactive plots directly within the notebook environment. This makes it easy to explore data, visualize results, and share findings with others.
- Object-Oriented Interface: Matplotlib provides an object-oriented interface for creating and customizing plots, allowing users to build complex plots by combining basic plot elements (e.g., axes, lines, markers, patches) in a modular and flexible manner.
- Matplotlib.pyplot Interface: Matplotlib also provides a MATLAB-style pyplot interface, which provides a convenient way to create simple plots quickly. This interface is particularly useful for interactive exploration and prototyping.
- Extensibility: Matplotlib is highly extensible and customizable, with a large ecosystem of third-party packages and toolkits that build on top of the core library. These packages provide additional functionality and specialized plotting capabilities for specific domains, such as seaborn for statistical visualization, mplfinance for financial plotting, and cartopy for geospatial plotting.
- Matplotlib Basemap Toolkit: Matplotlib includes the Basemap toolkit for plotting geographical data and maps. It provides a wide range of map projections and customization options for creating maps of different regions and spatial features.

3.6 Scikit-learn

Scikit-learn, often abbreviated as sklearn, is a widely-used Python library for machine learning. It is built on top of other popular scientific computing libraries such as NumPy, SciPy, and matplotlib. Scikit-learn provides simple and efficient tools for data mining and data analysis, with a focus on ease of use, code readability, and performance.

Fig 7. Scikit learn Library

Key features of scikit-learn include:

- Simple and Consistent API: Scikit-learn offers a uniform and easy-to-use API across different algorithms, making it straightforward to experiment with various machine learning models without needing to learn new syntax for each one.

- Wide Range of Algorithms: It includes implementations of a wide variety of supervised and unsupervised learning algorithms, including regression, classification, clustering, dimensionality reduction, and more. These algorithms cover a broad spectrum of machine learning tasks and can be applied to a wide range of datasets.
- Data Preprocessing: Scikit-learn provides tools for data preprocessing, including scaling, normalization, encoding categorical variables, handling missing values, and feature selection. These preprocessing techniques help prepare data for modeling and improve the performance of machine learning algorithms.
- Model Evaluation and Selection: It offers functions for evaluating and comparing the performance of machine learning models using various metrics such as accuracy, precision, recall, F1-score, and ROC curves. Cross-validation and hyperparameter tuning techniques are also available to assist in model selection and optimization.
- Integration with NumPy and Pandas: Scikit-learn seamlessly integrates with NumPy arrays and Pandas DataFrames, allowing users to work with data in familiar data structures and easily interface with other Python libraries for data manipulation and analysis.
- Feature Extraction and Transformation: It includes utilities for feature extraction and transformation, such as text feature extraction, image feature extraction, and feature scaling. These tools are essential for preprocessing and extracting meaningful information from raw data.
- Scalability and Performance: Scikit-learn is designed to be efficient and scalable, with support for parallel computing and optimized implementations of machine learning algorithms. It can handle large datasets and complex models while maintaining good performance.
- Extensive Documentation and Community Support: Scikit-learn has extensive documentation, tutorials, and examples to help users get started with machine learning tasks. It also has a large and active community of developers and users who contribute to its development, provide support, and share knowledge.

3.7 Python

Python is a high-level, general-purpose programming language known for its simplicity, readability, and versatility. It was created by Guido van Rossum and first released in 1991. Python emphasizes code readability and a clean syntax, which makes it easy to learn and use, especially for beginners.

Fig 8. Python

Here are some key features and characteristics of Python:

- Simple and Readable Syntax: Python's syntax is designed to be simple and easy to read, resembling pseudo-code. It uses indentation (whitespace) to define code blocks, which enhances code readability.
- Interpreted and Interactive: Python is an interpreted language, meaning that code is executed line by line, making it suitable for interactive use in REPL (Read-Eval-Print Loop) environments. This allows users to experiment with code and get immediate feedback.
- Multi-paradigm: Python supports multiple programming paradigms, including procedural, object-oriented, and functional programming styles. This flexibility allows developers to choose the most appropriate approach for their projects.
- Dynamic Typing and Automatic Memory Management: Python uses dynamic typing, meaning that variable types are determined at runtime. It also features automatic memory management through garbage collection, which simplifies memory allocation and deallocation.
- Large Standard Library: Python comes with a large and comprehensive standard library, providing a wide range of modules and packages for various tasks such as file I/O, networking, data manipulation, web development, and more. This eliminates the need for third-party libraries in many cases.
- Extensive Ecosystem: In addition to the standard library, Python has a vast ecosystem of third-party libraries and frameworks developed by the community. These libraries cover a wide range of domains, including scientific computing, data analysis, machine learning, web development, game development, and more.
- Cross-platform: Python is cross-platform, meaning that it runs on multiple operating systems, including Windows, macOS, and various Unix-like systems (e.g., Linux). This allows developers to write code once and run it anywhere without modification.
- Community and Support: Python has a large and active community of developers and users who contribute to its development, provide support, and share knowledge through forums, mailing lists, and online communities. This vibrant community is one of Python's greatest strengths.

3.8 NLTK

NLTK (Natural Language Toolkit) can be instrumental in various aspects of fake news detection during elections. Here's how it can be applied:

Fig. 9 NLTK

- Text Preprocessing: Before analyzing text data for fake news detection, preprocessing steps are crucial. NLTK offers tools for tokenization, removing stopwords, stemming, and lemmatization. By cleaning and normalizing the text data, NLTK helps in preparing it for further analysis.
- Feature Extraction: NLTK enables the extraction of relevant features from text data. These features can include word frequencies, n-grams, named entities, and parts of speech. By extracting meaningful features, NLTK aids in representing text data in a format suitable for machine learning algorithms.
- Sentiment Analysis: NLTK includes tools for sentiment analysis, which can be useful in assessing the sentiment or emotional tone of news articles, social media posts, and other textual data related to elections. Sentiment analysis can help identify biased or emotionally charged content, which may be indicative of fake news.
- Named Entity Recognition (NER): NLTK provides capabilities for named entity recognition, allowing identification and classification of named entities such as people, organizations, and locations in text data. NER can help detect key entities mentioned in news articles or social media posts, aiding in the identification of potential sources of fake news.
- Text Classification: NLTK supports text classification tasks, including techniques such as Naive Bayes classification and maximum entropy classification. Researchers can train text classification models using NLTK to classify news articles, social media posts, and other textual data as either genuine or fake based on features extracted from the text.
- Language Analysis: NLTK offers tools for analyzing the linguistic characteristics of text data, such as vocabulary richness, readability scores, and syntactic complexity. By analyzing the language patterns in news articles and social media posts, NLTK can help identify linguistic cues that may indicate the presence of fake news.

- Corpus Analysis: NLTK provides access to various text corpora and language resources, including datasets, lexicons, and linguistic resources. Researchers can use these resources to analyze language usage patterns, identify common themes or topics in news articles and social media discussions, and develop linguistic models for fake news detection.

By leveraging the capabilities of NLTK in text processing, analysis, and classification, researchers can develop robust methodologies for detecting and combating fake news during elections. Combined with other machine learning techniques and domain-specific knowledge, NLTK can play a significant role in identifying misinformation and promoting the integrity of electoral processes.

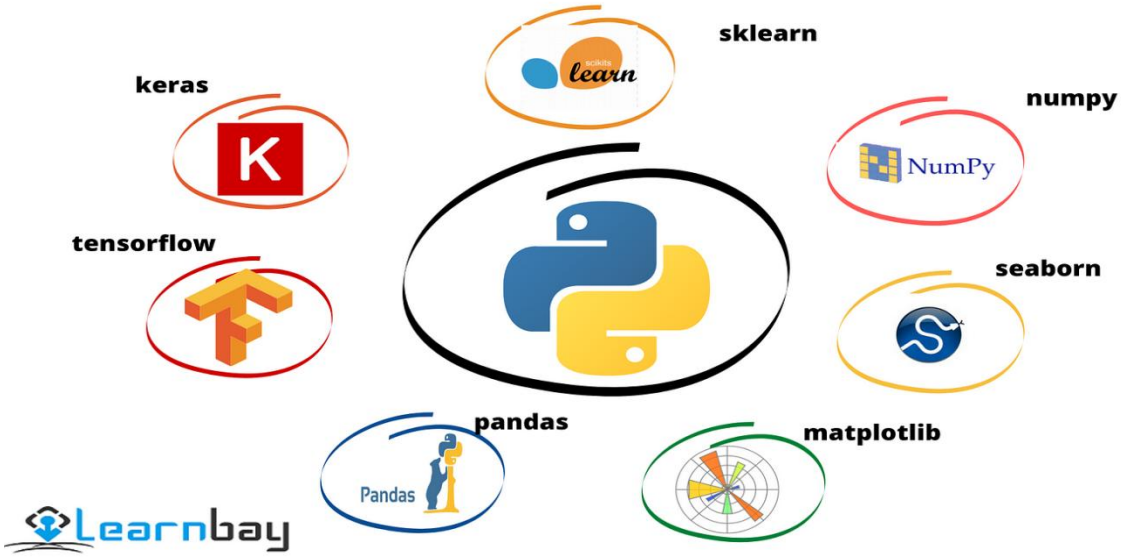


Fig 10. ML libraries

CHAPTER 4: MACHINE LEARNING ALGORITHMS

4.1 Logistic Regression

Logistic regression is a statistical method commonly used in the field of mental health to predict the presence or absence of a particular disorder based on one or more predictor variables. Unlike linear regression, which predicts continuous outcomes, logistic regression is specifically designed for binary outcomes, making it well-suited for classifying individuals into diagnostic categories.

In mental health disorder classification, logistic regression can be applied to various scenarios, such as predicting the likelihood of developing a disorder based on risk factors, identifying significant predictors of symptom severity, or assessing the effectiveness of interventions. By modeling the relationship between predictor variables and the probability of a binary outcome (e.g., presence or absence of the disorder), logistic regression provides insights into the factors associated with mental health disorders and aids in clinical decision-making.

Application of Logistic Regression in Mental Health Disorder Classification:

Identifying Risk Factors: Logistic regression can be used to identify risk factors associated with the development of mental health disorders. Researchers may collect data on demographic characteristics, genetic predispositions, environmental exposures, and psychosocial factors and use logistic regression to determine which variables are significant predictors of disorder onset.

Diagnostic Prediction: Logistic regression models can be developed to predict the likelihood of individuals belonging to different diagnostic categories based on their symptoms, clinical characteristics, and other relevant factors. These models can help clinicians make informed decisions about diagnosis and treatment planning.

Outcome Prediction: In longitudinal studies or clinical trials, logistic regression can be used to predict the likelihood of specific outcomes, such as treatment response, symptom remission, or relapse. By analyzing baseline predictors, clinicians can identify individuals at higher risk of poor outcomes and tailor interventions accordingly.

Screening and Early Detection: Logistic regression models can be employed for screening purposes to identify individuals at risk of developing mental health disorders or experiencing worsening symptoms. Screening tools may include self-report measures, clinical assessments, or biomarkers, and logistic regression can help prioritize individuals for further evaluation or intervention.

Evaluation of Interventions: Logistic regression is utilized to evaluate the effectiveness of interventions or treatment modalities in mental health. Researchers may compare outcomes between intervention and control groups while adjusting for potential confounding variables to assess the impact of the intervention on disorder prevalence or severity.

Overall, logistic regression plays a valuable role in mental health disorder classification by providing a statistical framework for understanding the relationships between predictor variables and diagnostic outcomes. Its flexibility, interpretability, and applicability to binary outcomes make it a versatile tool for researchers and clinicians working in the field of mental health.

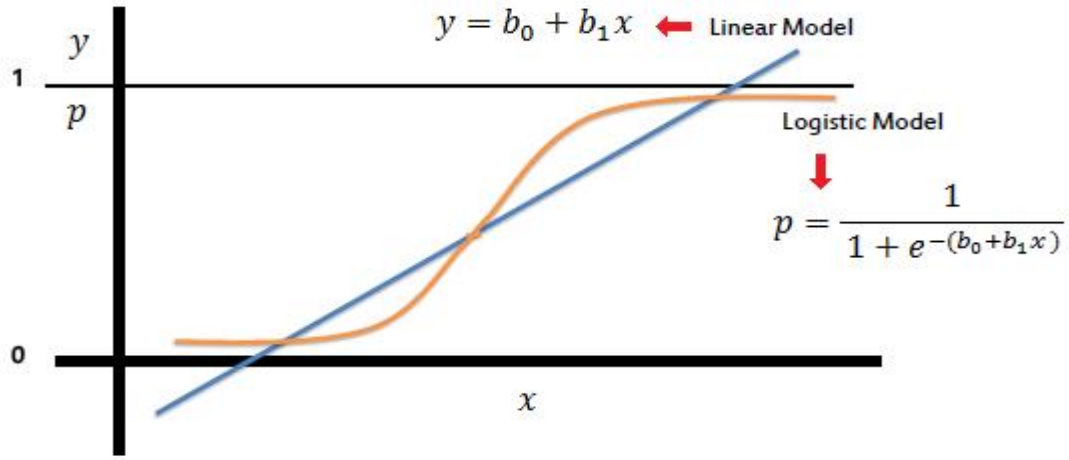


Fig 11. Logistic Regression

4.2 Random forest

Introduction to Random Forest:

Random Forest is an ensemble learning method that combines the predictions of multiple individual decision trees to improve accuracy and generalization. Each decision tree in the forest is trained on a random subset of the training data and a random subset of the features. During prediction, the individual trees "vote" on the class label, and the most common prediction becomes the final output. This ensemble approach helps reduce overfitting and increases robustness, making Random Forest particularly effective for complex classification tasks.

Application in Mental Health Disorder Classification:

Data Collection and Preparation:

Gather a diverse range of data related to mental health disorders, including demographic information, behavioral patterns, medical history, and psychological assessments.

Preprocess the data by handling missing values, encoding categorical variables, and normalizing numerical features.

Feature Selection and Engineering:

Identify relevant features that are predictive of mental health disorders. This might involve statistical analysis, domain expertise, or feature importance techniques specific to Random Forest.

Perform feature engineering to create new features or transform existing ones to enhance the predictive power of the model.

Model Training:

Split the dataset into training and testing sets (and possibly validation sets).

Train the Random Forest model on the training data, specifying the number of trees in the forest and other hyperparameters.

The algorithm builds multiple decision trees, each trained on a bootstrap sample of the data and considering only a random subset of features at each split.

Evaluation and Validation:

Evaluate the model's performance on the testing set using metrics such as accuracy, precision, recall, F1-score, and ROC-AUC.

Validate the model's robustness through techniques like cross-validation to ensure its generalization to unseen data and mitigate overfitting.

Interpretation and Insights:

Random Forest provides insights into feature importance, indicating which features contribute most to the classification task. This information can help understand the underlying factors associated with different mental health disorders.

Deployment and Monitoring:

Deploy the trained Random Forest model in real-world applications for mental health disorder classification, such as screening tools or decision support systems.

Monitor the model's performance over time, retraining it periodically with new data to maintain its effectiveness and adaptability to changing patterns in mental health data.

Random Forest's flexibility, robustness, and interpretability make it a valuable tool for mental health disorder classification, offering insights into predictive factors while achieving high accuracy in classification tasks.

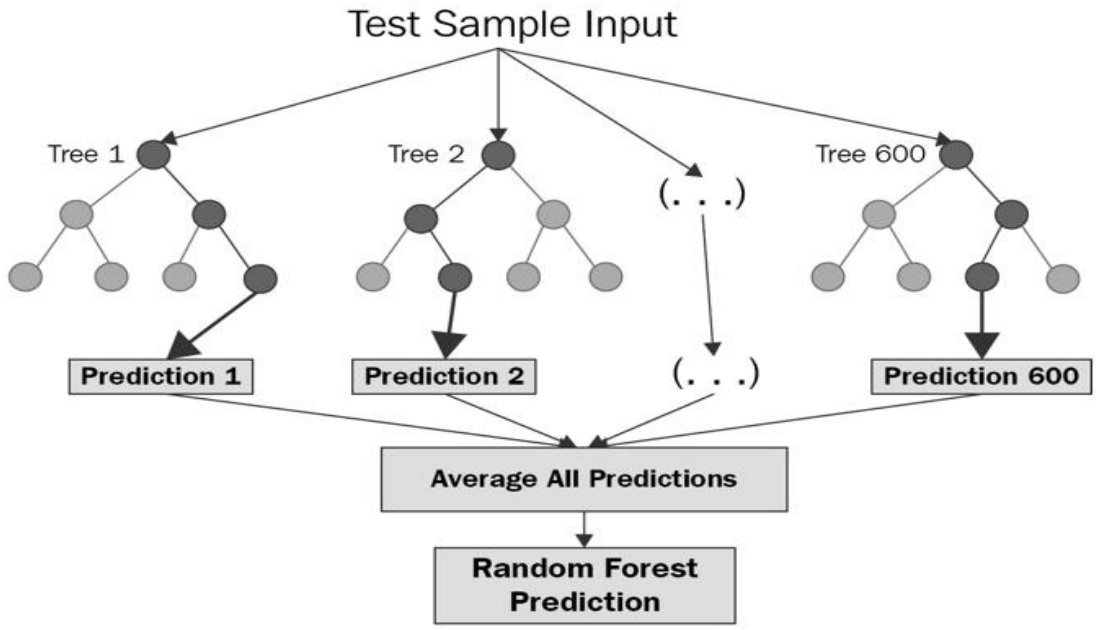


Fig 12. Random Forest

4.3 SUPPORT VECTOR MACHINE

Introduction to Support Vector Machines (SVMs):

SVMs are supervised learning models used for classification and regression analysis. They are particularly effective in high-dimensional spaces and are capable of handling both linear and non-linear data. The primary objective of SVMs is to find the optimal hyperplane that separates data points of different classes with the maximum margin, thus maximizing the classification performance.

Application of Support Vector Machines in Mental Health Disorder Classification:

Classification of Mental Health Disorders: SVMs can be used to classify individuals into different diagnostic categories based on their clinical features, demographic characteristics, or other relevant factors. For example, SVMs have been applied to neuroimaging data to distinguish between individuals with and without specific mental health disorders such as schizophrenia, depression, or anxiety disorders.

Prediction of Treatment Outcomes: SVMs can be employed to predict treatment outcomes for individuals with mental health disorders. By analyzing baseline characteristics and treatment response data, SVM models can identify factors associated with positive or negative treatment outcomes, helping clinicians tailor interventions to individual patient needs.

Identification of Biomarkers: SVMs have been used in conjunction with biological data (e.g., genetic markers, neuroimaging measures, physiological signals) to identify biomarkers associated with mental health disorders. These biomarkers can aid in early detection, diagnosis, and personalized treatment planning.

Integration of Multiple Data Modalities: SVMs can integrate information from multiple data modalities, such as clinical assessments, neuroimaging, and genetic data, to improve classification accuracy and provide a comprehensive understanding of mental health disorders. This multidimensional approach enables a more holistic assessment of individual patients and their unique clinical profiles.

Personalized Medicine: SVMs can facilitate personalized medicine approaches by identifying subgroups of individuals with distinct clinical characteristics or treatment responses. By stratifying patients based on SVM-derived classification models, clinicians can tailor interventions to specific subpopulations, optimizing treatment outcomes and minimizing adverse effects.

Risk Prediction and Prevention: SVMs can be used to predict the risk of developing mental health disorders or experiencing adverse outcomes (e.g., relapse, suicide attempts) based on known risk factors and longitudinal data. These predictive models enable early intervention and preventive strategies to mitigate the burden of mental illness on individuals and society.

Overall, SVMs offer a versatile and effective approach to mental health disorder classification, leveraging machine learning techniques to extract valuable insights from complex and heterogeneous data sources. By integrating SVMs into clinical practice and research, we can advance our understanding of mental health disorders and improve patient outcomes through personalized and evidence-based interventions.

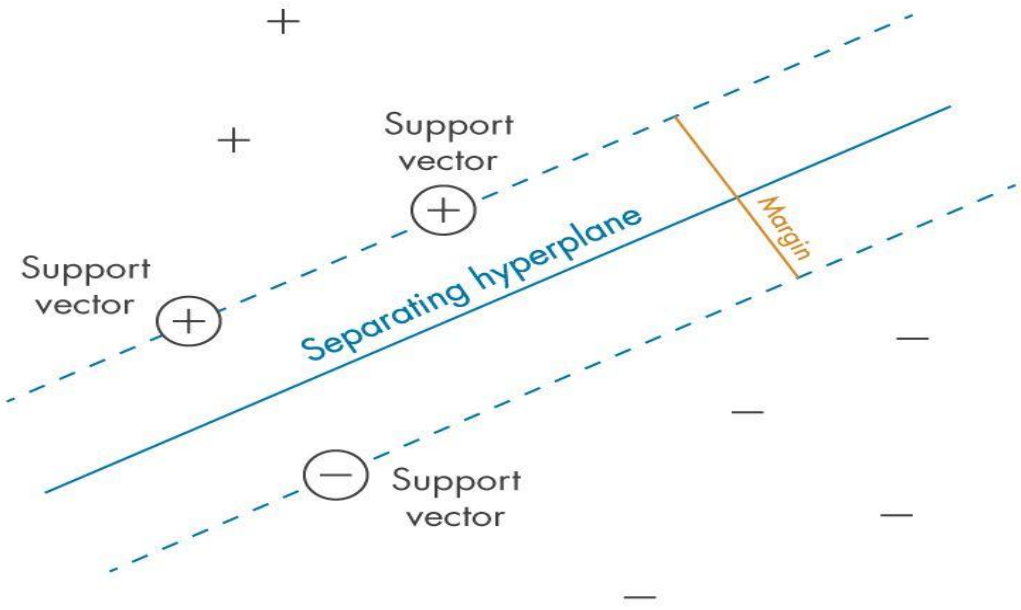


Fig 13. Support Vector Machine

4.4 Gradient Boosting Machine

Introduction to Gradient Boosting Machines (GBM):

Gradient Boosting Machines (GBM) are a type of ensemble learning method that builds a predictive model in a sequential manner by combining multiple weak learners, typically decision trees. The basic idea behind GBM is to iteratively train new models to correct the errors made by the previous ones, with each new model focusing on the residuals (i.e., the differences between the observed and predicted values) of the previous models. By optimizing a loss function (e.g., mean squared error for regression tasks, cross-entropy loss for classification tasks) using gradient descent, GBM gradually improves the predictive performance of the ensemble.

Application of Gradient Boosting Machines in Mental Health Disorder Classification:

High Predictive Accuracy:

GBM is known for its high predictive accuracy, making it well-suited for mental health disorder classification tasks where precision and reliability are paramount. By iteratively

refining the model to minimize prediction errors, GBM can capture complex patterns and relationships in the data, leading to more accurate predictions of diagnostic outcomes.

Handling Complex Data:

Mental health datasets often contain heterogeneous and high-dimensional data, including clinical assessments, neuroimaging scans, genetic markers, and behavioral measures. GBM is capable of handling such complex data structures and extracting meaningful information from diverse sources, allowing for comprehensive and integrative analyses of mental health disorders.

Feature Importance Analysis:

GBM provides a measure of feature importance, indicating the relative contribution of each predictor variable to the predictive performance of the model. In mental health disorder classification, feature importance analysis can help identify critical predictors or biomarkers associated with specific disorders, providing insights into the underlying mechanisms and risk factors.

Model Interpretability:

While GBM is inherently a complex model composed of multiple weak learners, techniques such as partial dependence plots, feature interaction analysis, and SHAP (SHapley Additive exPlanations) values can be used to interpret the model's predictions and understand the relative importance of different features. This interpretability is crucial for gaining insights into the factors driving mental health disorders and informing clinical decision-making.

Scalability and Efficiency:

GBM implementations, such as XGBoost and LightGBM, are optimized for scalability and efficiency, allowing for the training of large-scale models on extensive datasets in a reasonable amount of time. This scalability is particularly beneficial for mental health research, where datasets may be large and diverse, requiring robust and efficient modeling techniques.

Overall, Gradient Boosting Machines offer a powerful and versatile approach to mental health disorder classification, combining high predictive accuracy, flexibility, and interpretability. By leveraging the strengths of GBM, researchers and clinicians can develop more accurate and reliable models for diagnosing and understanding mental health disorders, ultimately leading to improved patient outcomes and personalized treatment strategies.

4.5 Ensemble model

Introduction to Ensemble Modeling:

Ensemble modeling involves combining the predictions of multiple base models to create a single, stronger model. The main idea behind ensemble modeling is that by aggregating the predictions of diverse models, the ensemble can achieve better performance than any individual model alone. Ensemble methods can be broadly categorized into two types: averaging methods and boosting methods.

Averaging Methods: Averaging methods combine the predictions of individual models by averaging their outputs. Examples include Bagging (Bootstrap Aggregating) and Random Forest, where multiple base models are trained independently, and their predictions are averaged to produce the final output.

Boosting Methods: Boosting methods sequentially train multiple weak learners, with each subsequent model focusing on the errors made by the previous ones. Examples include Gradient Boosting Machines (GBM) and AdaBoost, which iteratively improve the model's performance by adjusting the weights of misclassified instances.

Application of Ensemble Models in Mental Health Disorder Classification:

Improved Predictive Accuracy:

Ensemble models often outperform individual models by combining their strengths and mitigating their weaknesses. In mental health disorder classification, ensemble models can achieve higher predictive accuracy by leveraging diverse data sources, modeling techniques, and feature representations.

Robustness to Variability:

Ensemble models are more robust to variability in the data and less susceptible to overfitting than individual models. By aggregating predictions from multiple models, ensemble models can smooth out inconsistencies and generalize better to new, unseen data, enhancing their reliability and generalizability in real-world applications.

Feature Combination and Selection:

Ensemble models can effectively combine information from multiple features and feature representations, leading to better feature representation and discrimination. Ensemble methods can also perform implicit feature selection by weighting the importance of different features across multiple models, helping identify the most relevant predictors of mental health disorders.

Model Interpretability:

While ensemble models are inherently more complex than individual models, techniques such as feature importance analysis, model visualization, and model explanation methods can be used to interpret ensemble predictions and understand the factors driving mental health disorders. Ensemble models can provide valuable insights into the relationships between predictors and diagnostic outcomes, aiding clinical decision-making and hypothesis generation.

Integration of Heterogeneous Data:

Ensemble models are well-suited for integrating heterogeneous data sources, including clinical assessments, neuroimaging scans, genetic markers, and behavioral measures. By combining information from diverse data modalities, ensemble models can capture complementary aspects of mental health disorders and provide a more comprehensive understanding of their underlying mechanisms.

Overall, ensemble modeling offers a powerful and flexible approach to mental health disorder classification, enabling researchers and clinicians to leverage the collective intelligence of multiple models for improved prediction, interpretation, and decision-making. By harnessing the strengths of ensemble methods, we can advance our understanding of mental health disorders and develop more accurate and reliable diagnostic tools for personalized treatment and intervention strategies.

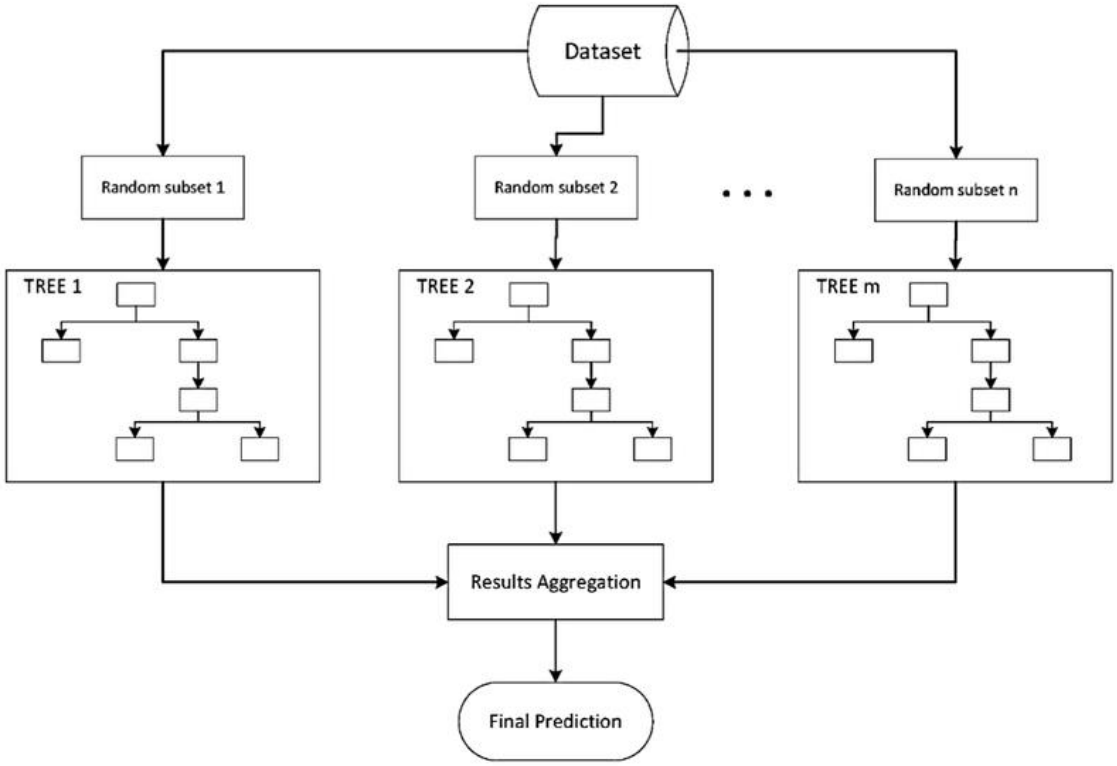


Fig 14. Ensemble Methods

CHAPTER 5: DATASET

Total Samples: 120

Features: 17

Target Variable (Dependent Variable): Mental Health Disorder (with four categories: normal, depression, bipolar-type-1, bipolar-type-2)

Features:

Sadness

Euphoric

Exhausted

Sleep Disorder

Mood Swing

Suicidal Thought

Anorexia

Authority Respect

Try-Explanation

Aggressive Response

Ignore & Move-On

Nervous Breakdown

Admit Mistakes

Overthinking

Sexual Activity

Concentration

Optimism

Target Variable (Mental Health Disorder):

Normal

Depression

Bipolar-type-1

Bipolar-type-2

5.1 Analysis and Modeling Considerations:

- Data Exploration: Before proceeding with modeling, it's essential to perform exploratory data analysis (EDA) to understand the distribution of features, identify any patterns or correlations, and gain insights into the characteristics of each mental health disorder category.
- Feature Importance: Given the nature of mental health disorders, some features may have more significant impacts on classification than others. Feature importance analysis can help prioritize features and understand their contributions to differentiating between disorder categories.
- Model Selection: With a dataset size of 121 samples and 17 features, it's crucial to choose a machine learning model that can handle relatively small datasets without overfitting. Random Forest, as previously discussed, is a suitable choice due to its ability to handle such datasets effectively.
- Model Evaluation: Evaluate the performance of the Random Forest model using appropriate metrics such as accuracy, precision, recall, F1-score, and confusion matrix. Additionally, consider techniques like cross-validation to ensure the model's robustness and generalization.
- Interpretability: Random Forest provides feature importance scores, allowing interpretation of which features contribute most to the classification of mental health disorders. This information can offer valuable insights into the underlying factors associated with different disorders.

Patient Number	Sadness	Euphoric	Exhausted
120 unique values	Usually 35% Sometimes 35% Other (36) 30%	Seldom 38% Sometimes 38% Other (29) 24%	Sometimes 32% Usually 28% Other (48) 40%
Patiant-01	Usually	Seldom	Sometimes
Patiant-02	Usually	Seldom	Usually
Patiant-03	Sometimes	Most-Often	Sometimes
Patiant-04	Usually	Seldom	Usually
Patiant-05	Usually	Usually	Sometimes
Patiant-06	Usually	Sometimes	Sometimes
Patiant-07	Seldom	Usually	Seldom
Patiant-08	Usually	Sometimes	Sometimes
Patiant-09	Most-Often	Seldom	Most-Often
Patiant-10	Usually	Seldom	Most-Often
Patiant-11	Seldom	Sometimes	Seldom
Patiant-12	Seldom	Sometimes	Sometimes
Patiant-13	Most-Often	Sometimes	Most-Often
Patiant-14	Usually	Usually	Sometimes
Patiant-15	Usually	Seldom	Most-Often
Patiant-16	Sometimes	Sometimes	Sometimes
Patiant-17	Sometimes	Usually	Sometimes
Patiant-18	Usually	Sometimes	Most-Often
Patiant-19	Most-Often	Seldom	Most-Often
Patiant-20	Sometimes	Usually	Usually
Patiant-21	Sometimes	Sometimes	Sometimes
Patiant-22	Usually	Sometimes	Sometimes

Fig.15. Dataset

CHAPTER 6: METHODOLOGY

6.1 Data collection and preparation

Data collection and preparation are crucial steps in any machine learning project, including mental health disorder classification. Here's a detailed overview of how to approach data collection and preparation specifically for this task:

1. Define Data Requirements:

Identify the target variable: In this case, it's the mental health disorder categories (e.g., normal, depression, bipolar-type-1, bipolar-type-2).

Determine the features: Select relevant features that might be indicative of mental health disorders. These could include demographic information, behavioral patterns, medical history, and psychological assessments.

Consider ethical considerations: Ensure that data collection methods comply with ethical guidelines, including obtaining informed consent, maintaining confidentiality, and protecting participants' privacy.

2. Data Collection:

Gather data from various sources: This could include clinical records, surveys, interviews, or online platforms.

Ensure data quality: Verify the accuracy and completeness of the collected data. Address any issues such as missing values, outliers, or inconsistencies.

3. Data Preprocessing:

Handle missing values: Decide on a strategy to deal with missing data, such as imputation, deletion, or using advanced techniques like mean imputation or regression imputation.

Encode categorical variables: Convert categorical variables into numerical representations using techniques like one-hot encoding or label encoding.

Normalize/Standardize numerical features: Scale numerical features to a similar range to prevent features with larger magnitudes from dominating the model's training process.

4. Exploratory Data Analysis (EDA):

Explore the distribution of features: Use histograms, box plots, and density plots to understand the distribution of each feature.

Analyze correlations: Examine correlations between features and the target variable to identify potential predictive relationships.

Visualize relationships: Create scatter plots or heatmaps to visualize relationships between different features and identify patterns.

5. Feature Engineering:

Create new features: Derive new features from existing ones that might better capture the underlying relationships in the data.

Transform variables: Apply transformations such as logarithmic transformation or polynomial transformation to improve linearity or reduce skewness in the data.

Feature scaling: Scale features to a similar range to ensure that no single feature dominates the model's learning process.

6. Data Splitting:

Split the dataset into training and testing sets: Typically, allocate a larger portion of the data (e.g., 80%) to training and a smaller portion (e.g., 20%) to testing.

Optionally, create a validation set: Reserve a portion of the training data as a validation set for hyperparameter tuning if needed.

7. Address Class Imbalance (if applicable):

If there is a significant class imbalance in the target variable (e.g., one class has much fewer samples than others), consider techniques such as oversampling, undersampling, or using algorithms that handle class imbalance effectively.

8. Data Augmentation (optional):

In some cases, especially if the dataset is small, data augmentation techniques such as generating synthetic samples or applying transformations to existing samples may be used to increase the dataset's size and diversity.

6.2 EDA

Exploratory Data Analysis (EDA) is a crucial step in understanding the underlying patterns and relationships within the dataset before building a machine learning model for mental health disorder classification. Here's how you can conduct EDA specifically for this task:

1. Overview of the Dataset:

Start by loading the dataset and examining its structure: number of samples, features, and target variable (mental health disorder categories).

Check for any missing values and outliers that may need to be addressed.

2. Distribution of Target Variable:

Visualize the distribution of the target variable (mental health disorder categories) using bar plots or pie charts.

Check for class imbalance, ensuring that each mental health disorder category has a reasonable number of samples for training the model effectively.

3. Analysis of Individual Features:

For each feature in the dataset, analyze its distribution across different mental health disorder categories.

Use histograms, box plots, or violin plots to visualize the distribution of numerical features.

For categorical features, create bar plots to show the frequency of each category within each mental health disorder category.

4. Correlation Analysis:

Examine the correlation between features and the target variable (mental health disorder categories).

Calculate correlation coefficients (e.g., Pearson correlation for numerical features, Cramer's V for categorical features) and visualize them using heatmaps or clustered correlation matrices.

Identify features that are strongly correlated with specific mental health disorder categories, as these may be important predictors.

5. Pairwise Relationships:

Explore pairwise relationships between features, especially those that show significant correlations with the target variable.

Create scatter plots or pair plots (for multiple features) to visualize relationships and identify any patterns or clusters that may exist.

6. Feature Importance:

If applicable, analyze feature importance scores obtained from preliminary models or feature selection techniques.

Determine which features have the most significant impact on predicting mental health disorder categories and prioritize them for further analysis.

7. Dimensionality Reduction (Optional):

Apply dimensionality reduction techniques such as Principal Component Analysis (PCA) or t-Distributed Stochastic Neighbor Embedding (t-SNE) to visualize high-dimensional data in lower-dimensional space.

Explore whether the data clusters or separates based on mental health disorder categories in the reduced dimensional space.

8. Insights and Interpretation:

Based on the findings from EDA, draw insights into the relationships between features and mental health disorder categories.

Identify potential predictive features or combinations of features that may be indicative of specific mental health disorders.

Use domain knowledge and clinical expertise to interpret the results and guide further analysis.

By conducting thorough exploratory data analysis, you can gain valuable insights into the dataset, identify important features, and understand the relationships between variables, laying the groundwork for building an effective mental health disorder classification model.

6.3 Feature selection and Engineering

Feature selection and engineering play a crucial role in building an effective machine learning model for mental health disorder classification. Here's how you can approach feature selection and engineering in this context:

1. Feature Selection:

Univariate Feature Selection: Use statistical tests (e.g., chi-square for categorical features, ANOVA for numerical features) to select features that have the strongest relationship with the target variable.

Feature Importance: Train a preliminary model (e.g., Random Forest) and analyze feature importance scores. Select the top-ranked features that contribute the most to the model's predictive performance.

Correlation Analysis: Identify features that are highly correlated with the target variable or with each other. Remove redundant features to reduce dimensionality and improve model efficiency.

Domain Knowledge: Consult with domain experts to identify features that are known to be relevant to mental health disorders. Incorporate expert knowledge into the feature selection process.

2. Feature Engineering:

Create New Features: Derive new features from existing ones that may capture additional information or patterns related to mental health disorders. For example:

Calculate aggregate statistics (e.g., mean, median, standard deviation) for numerical features over specific time periods.

Create interaction features by combining pairs of existing features (e.g., multiplying or dividing two features).

Transform Variables: Apply transformations to features to make the data more suitable for modeling. Common transformations include:

Logarithmic transformation for highly skewed numerical features.

Box-Cox transformation to stabilize variance and improve normality.

Encoding Categorical Variables: Convert categorical variables into numerical representations using techniques like one-hot encoding or label encoding.

Scaling Numerical Features: Scale numerical features to a similar range (e.g., using min-max scaling or standardization) to prevent features with larger magnitudes from dominating the model's learning process.

Handling Time-Series Data (if applicable): If the dataset includes temporal data, consider engineering features that capture trends, seasonality, or cyclic patterns over time.

3. Dimensionality Reduction (Optional):

If the dataset contains a large number of features, consider applying dimensionality reduction techniques such as Principal Component Analysis (PCA) or feature extraction methods to reduce the number of features while preserving the most relevant information.

Evaluate the trade-offs between model performance and interpretability when using dimensionality reduction techniques.

4. Iterative Process:

Feature selection and engineering should be treated as iterative processes that involve experimentation and refinement.

Evaluate the impact of feature selection and engineering on model performance using appropriate evaluation metrics (e.g., accuracy, precision, recall, F1-score).

Iterate on feature selection and engineering strategies based on the insights gained from model evaluation and domain knowledge.

By carefully selecting and engineering features, you can improve the predictive performance of your mental health disorder classification model and uncover valuable insights into the underlying factors associated with different disorders.

6.4 Model Selection and Training

Model selection and training are critical steps in building a machine learning model for mental health disorder classification. Here's how you can approach model selection and training in this context:

1. Choose Suitable Algorithms:

Consider algorithms that are well-suited for classification tasks and can handle both numerical and categorical data effectively.

Commonly used algorithms for classification include:

Random Forest

Support Vector Machines (SVM)

Logistic Regression

Gradient Boosting Machines (e.g., XGBoost, LightGBM)

Neural Networks (e.g., Multi-layer Perceptron)

Evaluate the strengths and weaknesses of each algorithm based on factors such as interpretability, scalability, and computational efficiency.

2. Initial Model Selection:

Start with a baseline model to establish a benchmark for comparison. This could be a simple algorithm like Logistic Regression or a decision tree.

Train and evaluate multiple algorithms using default parameters to compare their performance on the dataset.

Consider using cross-validation to assess each model's generalization performance and mitigate overfitting.

3. Hyperparameter Tuning:

Fine-tune the hyperparameters of the selected algorithms to optimize their performance.

Use techniques like grid search or random search to explore the hyperparameter space and identify the best combination of parameters.

Tune parameters such as learning rate, regularization strength, tree depth, and number of estimators based on the characteristics of the dataset and the chosen algorithm.

4. Model Training:

Split the dataset into training and testing sets (e.g., 80% training, 20% testing) to train and evaluate the models.

Ensure that the training set is representative of the overall dataset, and the testing set is held out for unbiased evaluation.

Train the selected algorithms on the training data using the optimized hyperparameters.

5. Model Evaluation:

Evaluate the trained models on the testing set using appropriate evaluation metrics for classification tasks.

Common evaluation metrics include accuracy, precision, recall, F1-score, and ROC-AUC.

Analyze the confusion matrix to understand the model's performance across different mental health disorder categories.

6. Compare Models:

Compare the performance of the trained models based on their evaluation metrics.

Select the model that achieves the highest performance on the testing set while considering factors such as interpretability, computational efficiency, and scalability.

7. Model Interpretability (Optional):

Consider the interpretability of the selected model, especially in sensitive domains like mental health.

Models like Logistic Regression and decision trees are more interpretable compared to complex models like neural networks or ensemble methods.

8. Additional Considerations:

Ensure ethical considerations and data privacy throughout the model selection and training process, especially when dealing with sensitive health data.

Document the chosen model's architecture, hyperparameters, and performance metrics for reproducibility and future reference.

By following these steps, you can effectively select and train a machine learning model for mental health disorder classification, providing valuable insights and support for clinical decision-making and intervention strategies.

6.5 Model Evaluation

Model evaluation is a crucial step in assessing the performance of a machine learning model for mental health disorder classification. Here's how you can effectively evaluate the model:

1. Splitting the Data:

Split the dataset into training and testing sets (e.g., 80% training, 20% testing) to train and evaluate the model.

Ensure that the splitting preserves the distribution of mental health disorder categories in both sets to maintain representativeness.

2. Evaluation Metrics:

Choose appropriate evaluation metrics for classification tasks. Common metrics include:

Accuracy: Measures the overall correctness of the model's predictions.

Precision: Measures the proportion of true positive predictions among all positive predictions.

Recall (Sensitivity): Measures the proportion of true positive predictions among all actual positive instances.

F1-score: Harmonic mean of precision and recall, providing a balanced measure of the model's performance.

ROC-AUC: Area under the Receiver Operating Characteristic (ROC) curve, measuring the model's ability to distinguish between classes.

Confusion Matrix: Provides a detailed breakdown of the model's predictions across different classes.

3. Evaluation Procedure:

Train the model on the training set using the optimized hyperparameters.

Evaluate the trained model on the testing set using the selected evaluation metrics.

Analyze the model's performance across different mental health disorder categories to identify any class-specific performance differences.

4. Interpretation and Analysis:

Interpret the evaluation metrics to understand the strengths and weaknesses of the model.

Identify areas where the model performs well and areas where it may need improvement.

Analyze the confusion matrix to understand the types of errors made by the model (e.g., false positives, false negatives) and their implications for mental health diagnosis.

5. Cross-Validation (Optional):

Consider using cross-validation techniques (e.g., k-fold cross-validation) to assess the model's generalization performance.

Perform multiple rounds of training and evaluation on different subsets of the data to obtain more robust performance estimates.

6. Comparison with Baseline:

Compare the performance of the trained model with a baseline model (e.g., a simple algorithm like Logistic Regression or a majority class classifier).

Assess whether the improvement in performance achieved by the model is statistically significant and clinically meaningful.

7. Sensitivity Analysis:

Perform sensitivity analysis to evaluate the robustness of the model to variations in hyperparameters or input data.

Assess how changes in key parameters (e.g., threshold for classification, feature selection criteria) impact the model's performance.

8. Ethical Considerations:

Ensure that the model's evaluation process complies with ethical guidelines and data privacy regulations, especially when dealing with sensitive health data.

Consider potential biases in the dataset and their impact on model performance, and take steps to mitigate them if necessary.

By following these steps, you can comprehensively evaluate the performance of a machine learning model for mental health disorder classification, gaining insights into its effectiveness and guiding further refinement and improvement efforts.

6.6 Interpretation and Insights

Interpretation and insights are essential aspects of any machine learning model, especially in sensitive domains like mental health disorder classification. Here's how you can interpret the model's predictions and derive insights from the classification results:

1. Feature Importance:

Analyze the feature importance scores provided by the model (e.g., Random Forest) to understand which features contribute most to the classification of mental health disorders.

Identify the top-ranked features and their respective importance levels, indicating their influence on predicting different disorder categories.

2. Clinical Relevance:

Interpret the results in the context of clinical knowledge and domain expertise. Consult with mental health professionals to validate the model's predictions and understand the clinical implications.

Identify features that align with known risk factors, symptoms, or diagnostic criteria for specific mental health disorders.

3. Patterns and Relationships:

Explore patterns and relationships between features and mental health disorder categories revealed by the model.

Identify associations between certain features and specific disorder categories, providing insights into potential predictive factors.

4. Misclassifications and Errors:

Analyze instances of misclassifications to understand the types of errors made by the model.

Investigate false positives (instances incorrectly classified as positive) and false negatives (instances incorrectly classified as negative) to identify potential areas for model improvement.

5. Class Imbalance and Bias:

Consider the impact of class imbalance and bias on the model's predictions and interpretations.

Evaluate whether the model exhibits biases towards certain mental health disorder categories or demographic groups and take steps to mitigate them if necessary.

6. Clinical Decision Support:

Use the model's predictions as decision support tools to aid clinicians in diagnosing and treating mental health disorders.

Provide explanations and justifications for the model's predictions to enhance its interpretability and trustworthiness in clinical practice.

7. Validation and Feedback:

Validate the model's predictions with real-world data and clinical observations to ensure its reliability and accuracy.

Solicit feedback from mental health professionals and stakeholders to refine the model and improve its clinical utility.

8. Ethical Considerations:

Consider ethical implications such as data privacy, informed consent, and potential biases throughout the interpretation process.

Ensure transparency and accountability in model development and deployment to maintain trust and integrity in mental health disorder classification.

By carefully interpreting the model's predictions and deriving meaningful insights, you can enhance the clinical utility and effectiveness of machine learning models for mental health disorder classification, ultimately improving patient outcomes and well-being.

6.7 Model Optimization and fine-tuning

Model optimization and fine-tuning are critical steps in improving the performance and effectiveness of a machine learning model for mental health disorder classification. Here's how you can approach model optimization and fine-tuning in this context:

1. Hyperparameter Tuning:

Identify the hyperparameters of the chosen algorithm (e.g., Random Forest) that significantly impact its performance.

Use techniques like grid search or random search to systematically explore the hyperparameter space and identify the optimal combination of parameters.

Tune hyperparameters such as:

Number of trees (n_estimators)

Maximum depth of trees (max_depth)

Minimum number of samples required to split a node (min_samples_split)

Minimum number of samples required to be at a leaf node (min_samples_leaf)

Maximum number of features to consider for each split (max_features)

2. Cross-Validation:

Employ cross-validation techniques (e.g., k-fold cross-validation) to assess the model's performance on different subsets of the data.

Use cross-validation to validate the effectiveness of hyperparameter tuning and ensure the model's generalization performance.

3. Regularization:

Apply regularization techniques to prevent overfitting and improve the model's generalization performance.

Adjust regularization parameters such as alpha (for L1 and L2 regularization) or gamma (for tree-based models) to control the model's complexity.

4. Feature Selection and Engineering:

Continuously evaluate and refine feature selection and engineering techniques to improve the model's predictive performance.

Experiment with different feature selection methods (e.g., univariate feature selection, feature importance analysis) and feature engineering techniques (e.g., creating new features, transforming variables) to enhance the model's ability to capture relevant information.

5. Ensemble Methods:

Explore ensemble methods to further improve the model's performance and robustness.

Consider techniques such as bagging (e.g., Random Forest), boosting (e.g., Gradient Boosting Machines), or stacking to combine multiple models and leverage their strengths.

6. Model Interpretability:

Ensure that the optimized model remains interpretable, especially in sensitive domains like mental health disorder classification.

Balance the trade-off between model complexity and interpretability, and prioritize models that strike a good balance between the two.

7. Validation and Testing:

Validate the performance of the optimized model using holdout validation or cross-validation on independent datasets.

Evaluate the model's performance on a separate test set to ensure unbiased assessment and verify its effectiveness in real-world scenarios.

8. Iterative Refinement:

Treat model optimization as an iterative process and continue to refine the model based on feedback and evaluation results.

Monitor the model's performance over time and re-evaluate its effectiveness periodically with new data.

6.8 Deployment and monitoring

Deployment and monitoring are crucial phases in the lifecycle of a machine learning model for mental health disorder classification. Here's how you can approach deployment and monitoring effectively:

1. **Deployment**:

Infrastructure Setup:

Prepare the necessary infrastructure for deploying the model, including computing resources, storage, and networking capabilities.

Choose an appropriate deployment environment, such as on-premises servers, cloud platforms (e.g., AWS, Azure, Google Cloud), or edge devices.

Model Deployment:

Deploy the trained model into the production environment using deployment frameworks or platforms (e.g., Flask, Django, Docker, Kubernetes).

Ensure that the deployment process is seamless and well-documented to facilitate integration with existing systems and workflows.

API Development:

Expose the model through a well-defined API (Application Programming Interface) to enable easy access and interaction with other applications and services.

Design the API endpoints for model inference, allowing input data to be submitted, and predictions to be retrieved in a standardized format (e.g., JSON).

Scalability and Performance:

Optimize the deployed model for scalability and performance to handle varying levels of workload and concurrent requests.

Implement load balancing, caching mechanisms, and auto-scaling strategies to ensure responsiveness and reliability under heavy traffic.

Security and Compliance:

Implement robust security measures to protect sensitive health data and ensure compliance with regulations (e.g., HIPAA, GDPR).

Encrypt data transmissions, enforce access controls, and monitor for potential security vulnerabilities regularly.

2. **Monitoring**:

Health Monitoring:

Implement monitoring solutions to continuously monitor the health and performance of the deployed model.

Monitor system resources (e.g., CPU, memory, disk usage) to ensure optimal performance and detect anomalies or resource constraints.

Set up alerts and notifications to alert administrators of any issues or abnormalities in real-time.

Model Performance Monitoring:

Monitor the performance metrics of the deployed model, such as inference latency, throughput, and error rates.

Track the distribution of input data and model predictions over time to detect drifts or shifts in data patterns that may affect model performance.

Data Quality Monitoring:

Monitor the quality and consistency of input data to ensure that it meets the model's expectations and requirements.

Implement data validation checks and data quality monitoring pipelines to detect data anomalies, missing values, or data inconsistencies.

Feedback Loop and Model Updates:

Establish a feedback loop to collect feedback from end-users, clinicians, and stakeholders regarding the model's performance and usability.

Use the feedback to iteratively improve the model by incorporating new data, retraining the model, or updating its parameters as needed.

Compliance Monitoring:

Regularly audit the deployed model to ensure compliance with regulatory requirements and ethical guidelines.

Maintain comprehensive documentation of model updates, data sources, and model decisions to support compliance audits and regulatory reporting.

Continuous Improvement:

Iterative Development:

Continuously iterate on the deployed model based on feedback, monitoring insights, and evolving requirements.

Incorporate new features, update model parameters, and retrain the model with fresh data to adapt to changing patterns and needs.

Collaboration and Communication:

Foster collaboration between data scientists, developers, clinicians, and stakeholders to ensure alignment with business goals and clinical objectives.

Establish clear channels of communication and feedback mechanisms to facilitate continuous improvement and knowledge sharing.

Documentation and Knowledge Management:

Maintain comprehensive documentation of the deployment process, model architecture, monitoring procedures, and performance metrics.

Create knowledge repositories and share best practices to enable effective collaboration, troubleshooting, and onboarding of new team members.

By following these steps, you can deploy and monitor a machine learning model for mental health disorder classification effectively, ensuring its reliability, scalability, and compliance with regulatory requirements, while also enabling continuous improvement and innovation.

6.9 WORKING OF MHDC:

The overview you provided offers a comprehensive understanding of Linear Regression (LR), Random Forest (RF), Support Vector Machine (SVM), ensemble models, dataset details, and the steps involved in training data, preprocessing, building an ensemble model, training & evaluation, and testing & prediction in machine learning. Linear Regression (LR), Random Forest (RF), and Support Vector Machine (SVM) are powerful supervised machine learning algorithms used for various classification and regression tasks. Ensemble models combine multiple individual models to improve predictive performance and robustness. The dataset contains four types of mental health disorder datasets: normal, depression, bipolar-type 1, and bipolar type 2. It consists of 17 features related to mental health, including Sadness, Euphoric, Exhausted, Sleep Disorder, and others. The dataset comprises 121 samples for analysis. The dataset contains four types of mental health disorder datasets: normal, depression, bipolar-type 1, and bipolar type 2. It consists of 17 features related to mental health, including Sadness, Euphoric, Exhausted, Sleep Disorder, and others. The dataset comprises 121 samples for analysis.

6.10 Analyzing the System:- Analyzing the system involves assessing the performance of the machine learning models, identifying any issues or limitations, and making improvements as necessary. This may include analyzing metrics such as accuracy, precision, recall, F1 score, mean squared error, and R-squared to evaluate model performance. Feature importance analysis can be performed to understand the contribution of each feature to the model's predictions. Model interpretation techniques, such as SHAP values or partial dependence plots, can provide insights into how the models make predictions. Each step is crucial in ensuring the effectiveness, reliability, and generalization ability of the models for various tasks and domains. Analyzing the system involves rigorous evaluation and interpretation of the model's performance and behavior to make informed decisions and improvements.

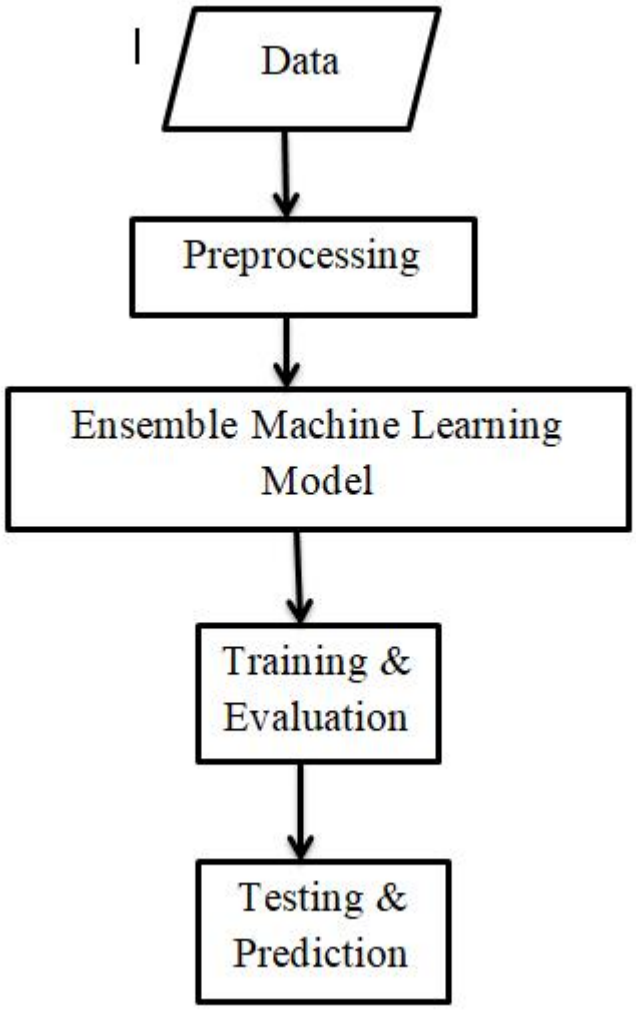


Fig .16. Steps involved in MHDC

CHAPTER 7: IMPLEMENTATION

```
import pandas
orginal_df=pandas.read_csv('/content/drive/MyDrive/MHDC/Orginal-Dataset-Mental-Disorders (1).csv')
cleaned_df=pandas.read_csv('/content/drive/MyDrive/MHDC/Cleaned-Dataset-Mental-Disorder.csv')
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
orginal_df.head(10)
```

	Patient Number	Sadness	Euphoric	Exhausted	Sleep dissorder	Mood Swing	Suicidal thoughts	Anorxia	Authority Respect	Try-Explanation	Aggressive Response	Ignore & Move-On	Nervous Break-down	Admit Mistakes	Overthinking	Sexual Activity	Concentration	Optimisim	Expert Diagnose
0	Patiant-01	Usually	Seldom	Sometimes	Sometimes	YES	YES	NO	NO	YES	NO	NO	YES	YES	YES	3 From 10	3 From 10	4 From 10	Bipolar Type-2
1	Patiant-02	Usually	Seldom	Usually	Sometimes	NO	YES	NO	NO	NO	NO	NO	NO	NO	NO	4 From 10	2 From 10	5 From 10	Depression
2	Patiant-03	Sometimes	Most-Often	Sometimes	Sometimes	YES	NO	NO	NO	YES	YES	NO	YES	YES	NO	6 From 10	5 From 10	7 From 10	Bipolar Type-1
3	Patiant-04	Usually	Seldom	Usually	Most-Often	YES	YES	YES	NO	YES	NO	NO	NO	NO	NO	3 From 10	2 From 10	2 From 10	Bipolar Type-2
4	Patiant-05	Usually	Usually	Sometimes	Sometimes	NO	NO	NO	NO	NO	NO	NO	YES	YES	YES	5 From 10	5 From 10	6 From 10	Normal
5	Patiant-06	Usually	Sometimes	Sometimes	Most-Often	NO	YES	YES	YES	NO	NO	NO	NO	YES	NO	3 From 10	5 From 10	5 From 10	Depression
6	Patiant-07	Seldom	Usually	Seldom	Sometimes	YES	YES	YES	NO	YES	YES	NO	YES	YES	YES	7 From 10	2 From 10	9 From 10	Bipolar Type-1
7	Patiant-08	Usually	Sometimes	Sometimes	Sometimes	NO	NO	NO	NO	YES	NO	NO	NO	NO	YES	5 From 10	5 From 10	5 From 10	Normal
8	Patiant-09	Most-Often	Seldom	Most-Often	Usually	YES	YES	YES	NO	YES	YES	NO	YES	NO	NO	8 From 10	2 From 10	3 From 10	Bipolar Type-2
9	Patiant-10	Usually	Seldom	Most-Often	Sometimes	NO	NO	NO	NO	YES	NO	NO	YES	YES	YES	3 From 10	4 From 10	2 From 10	Depression

```
df.replace({'Bipolar Type-2':0,'Depression':1,'Bipolar Type-1':2,'Normal':3},inplace=True)
```

```
df.replace({'3 From 10':3,'2 From 10':2,'6 From 10':6,'5 From 10':5,'1 From 10':1,'7 From 10':7,'4 From 10':4,'8 From 10':8,'9 From 10':9},inplace=True)
```

```
df['Admit Mistakes']=[1 if val =='YES' else 0 for val in df['Admit Mistakes']]
```

```
df['Anorxia']=[1 if val =='YES' else 0 for val in df['Anorxia']]
```

```
df['Mood Swing']=[1 if val =='YES' else 0 for val in df['Mood Swing']]
```

```
df['Suicidal thoughts']=[1 if val == 'YES' else 0 for val in df['Suicidal thoughts']]
```

```
df['Authority Respect']=[1 if val == 'YES' else 0 for val in df['Authority Respect']]
```

```
df['Try-Explanation']=[1 if val == 'YES' else 0 for val in df['Try-Explanation']]
```

```
df['Aggressive Response']=[1 if val == 'YES' else 0 for val in df['Aggressive Response']]
```

```
df['Ignore & Move-On']=[1 if val == 'YES' else 0 for val in df['Ignore & Move-On']]
```

```
df['Nervous Break-down']=[1 if val == 'YES' else 0 for val in df['Nervous Break-down']]
```

```
df['Overthinking']=[1 if val == 'YES' else 0 for val in df['Overthinking']]
```

After cleaning Dataset:

```
df.head(10)
```

	Sadness	Euphoric	Exhausted	Sleep dissorder	Mood Swing	Suicidal thoughts	Anorxia	Authority Respect	Try-Explanation	Aggressive Response	Ignore & Move-On	Nervous Break-down	Admit Mistakes	Overthinking	Sexual Activity	Concentration	Optimisim	Expert Diagnose
0	0.8	1.9	0.6	0.6	1	0	0	0	1	0	0	1	1	1	3	3	4	0
1	0.8	1.9	0.8	0.6	0	1	0	0	0	0	0	0	0	0	4	2	5	1
2	0.6	1.5	0.6	0.6	1	0	0	0	1	1	0	1	1	0	6	5	7	2
3	0.8	1.9	0.8	1.5	1	1	1	0	1	0	0	0	0	0	3	2	2	0
4	0.8	0.8	0.6	0.6	0	0	0	0	0	0	0	1	1	1	5	5	6	3
5	0.8	0.6	0.6	1.5	0	1	1	1	0	0	0	0	1	0	3	5	5	1
6	1.9	0.8	1.9	0.6	1	1	1	0	1	1	0	1	1	1	7	2	9	2
7	0.8	0.6	0.6	0.6	0	0	0	0	1	0	0	0	0	1	5	5	5	3
8	1.5	1.9	1.5	0.8	1	1	1	0	1	1	0	1	0	0	8	2	3	0
9	0.8	1.9	1.5	0.6	0	0	0	0	1	0	0	1	1	1	3	4	2	1

To predict mental disorders based on symptoms using machine learning with 17 features and 120 samples, follow these steps:

Data Collection: Assemble a diverse dataset with symptom information and diagnosed mental disorders.

Data Preprocessing: Clean, handle missing values, and encode categorical variables. Split data into symptom features and diagnosed disorder targets.

Feature Engineering: Analyze feature relevance and perform dimensionality reduction if necessary.

Model Selection: Choose appropriate classification algorithms (e.g., Random Forest, SVM,LR) and ensemble methods.

Training and Evaluation: Split data, train models on training set, and evaluate using metrics like accuracy, precision, recall, and F1-score.

Validation: Validate models with techniques such as cross-validation to ensure robustness.

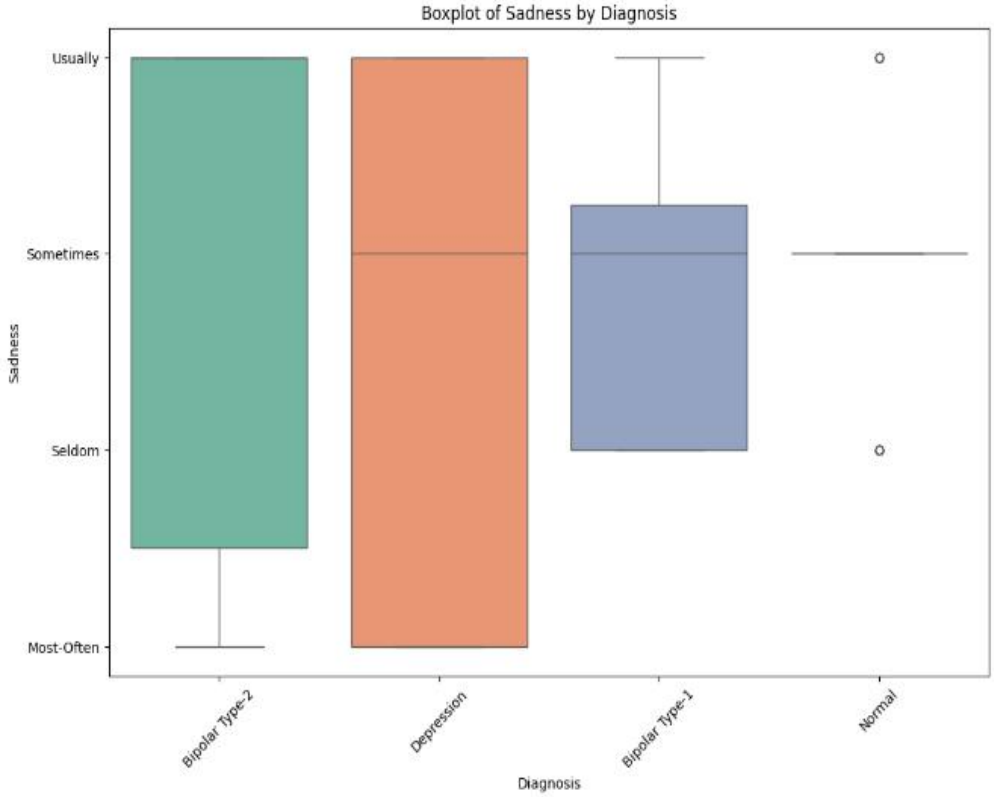
Prediction: Deploy the best-performing model for predicting mental disorders based on symptoms.

Monitoring and Updating: Continuously monitor model performance and update as new data or understanding of mental disorders emerges.

Overall, the process of training data, preprocessing, building an ensemble model, and training & evaluation, followed by testing & prediction, forms a structured approach to developing and deploying machine learning models for various tasks and domains. Each step is crucial in ensuring the model's effectiveness, reliability, and generalization ability.

```
# Boxplot of Diagnoses vs. Feature
plt.figure(figsize=(12, 8))
sns.boxplot(x='Expert Diagnose', y='Sadness', data=data, palette='Set2')
plt.title('Boxplot of Sadness by Diagnosis')
plt.xlabel('Diagnosis')
plt.ylabel('Sadness')
plt.xticks(rotation=45)
plt.show()
```

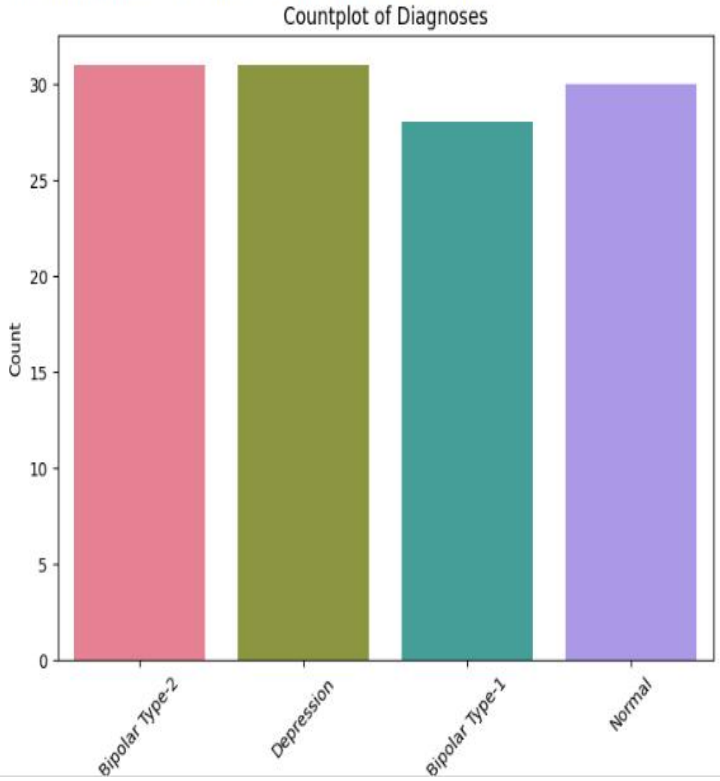
```
<ipython-input-7-c29b017f4603>:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.boxplot(x='Expert Diagnose', y='Sadness', data=data, palette='Set2')
```



```
import matplotlib.pyplot as plt
import seaborn as sns

data = orginal_df
```

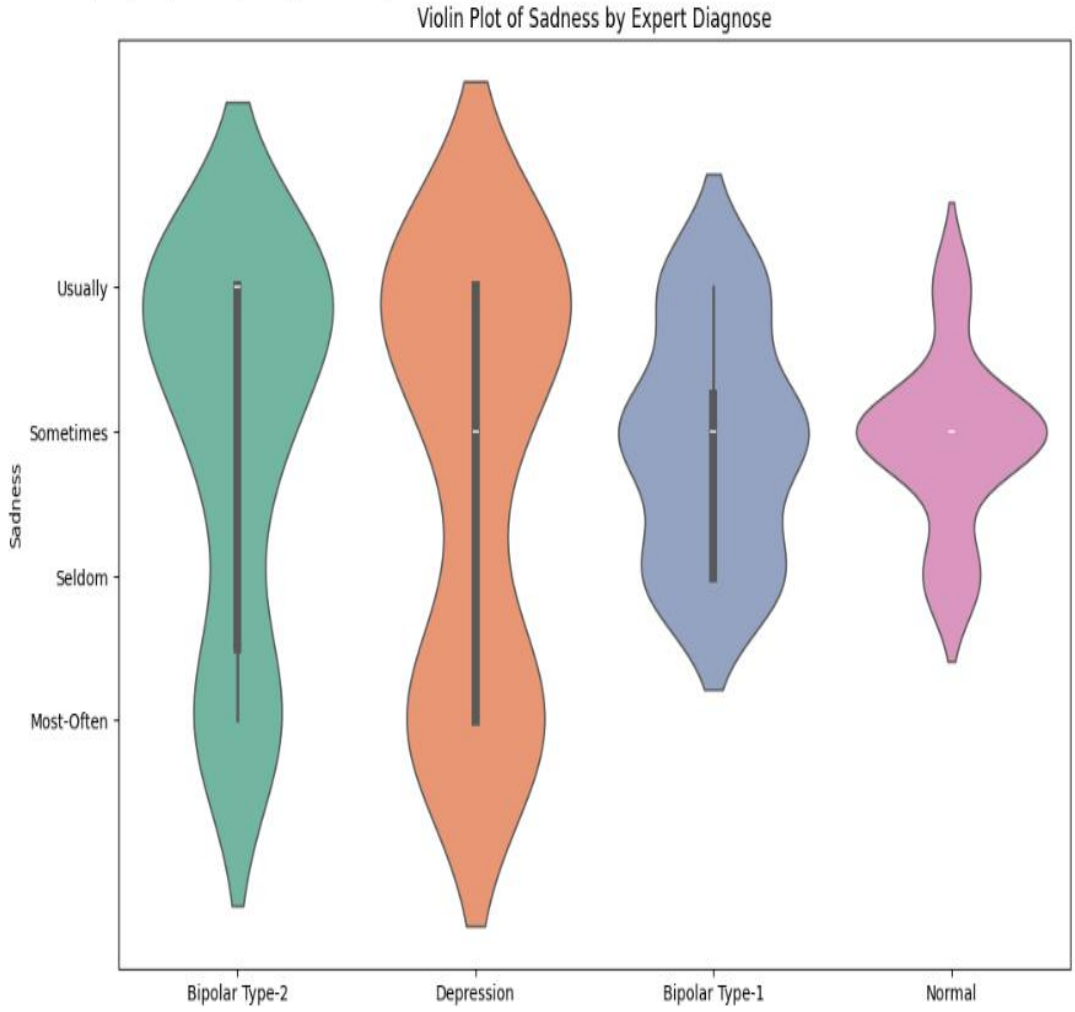
```
# Countplot of Diagnosis
plt.figure(figsize=(8, 6))
sns.countplot(x='Expert Diagnose', data=data, palette='husl')
plt.title('Countplot of Diagnoses')
plt.xlabel('Diagnoses')
plt.ylabel('Count')
plt.xticks(rotation=45)
plt.show()
```

```
<ipython-input-6-c4db4bbc8db7>:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.countplot(x='Expert Diagnose', data=data, palette='husl')
```



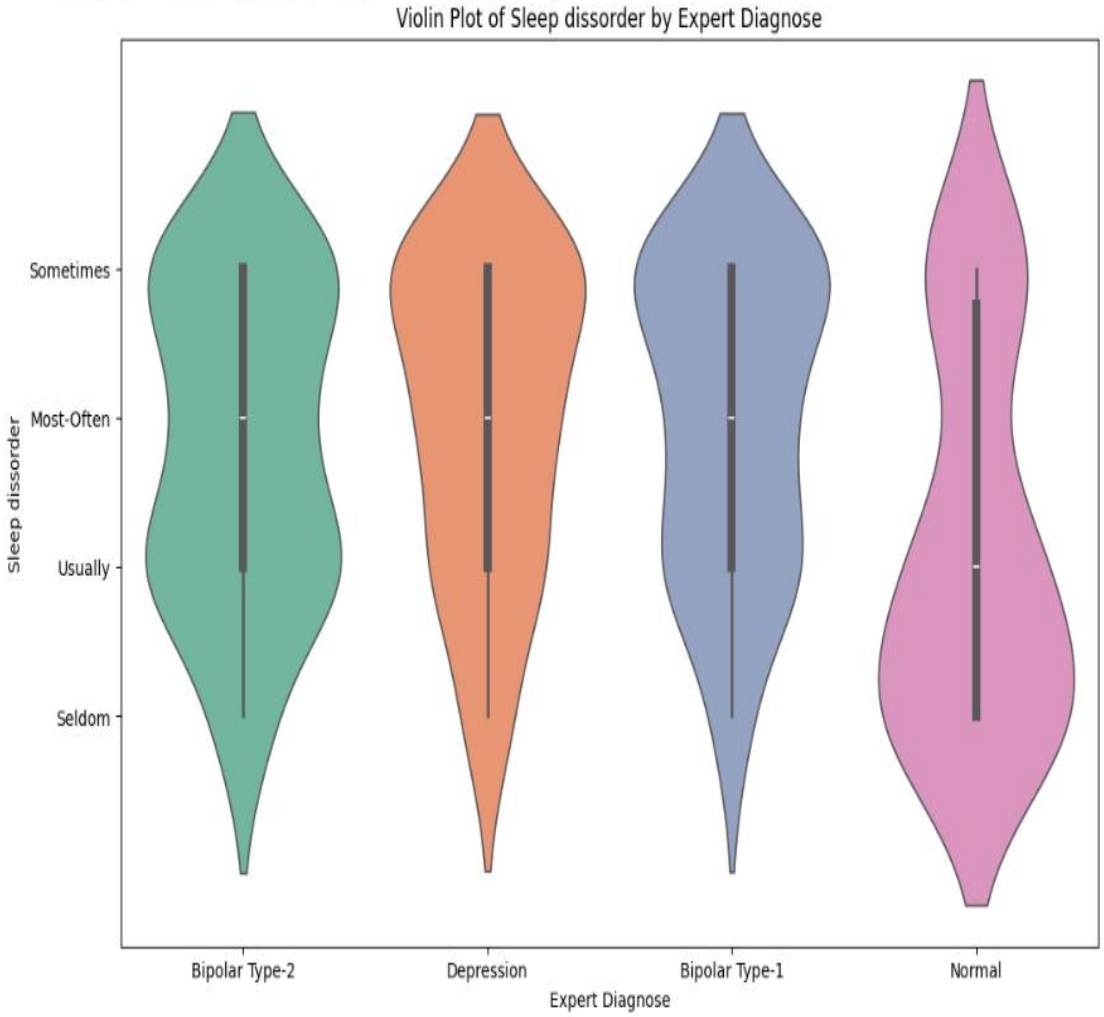
```
# Violin Plot
plt.figure(figsize=(12, 8))
sns.violinplot(x='Expert Diagnose', y='Sadness', data=data, palette='Set2')
plt.title('Violin Plot of Sadness by Expert Diagnose')
plt.show()
```

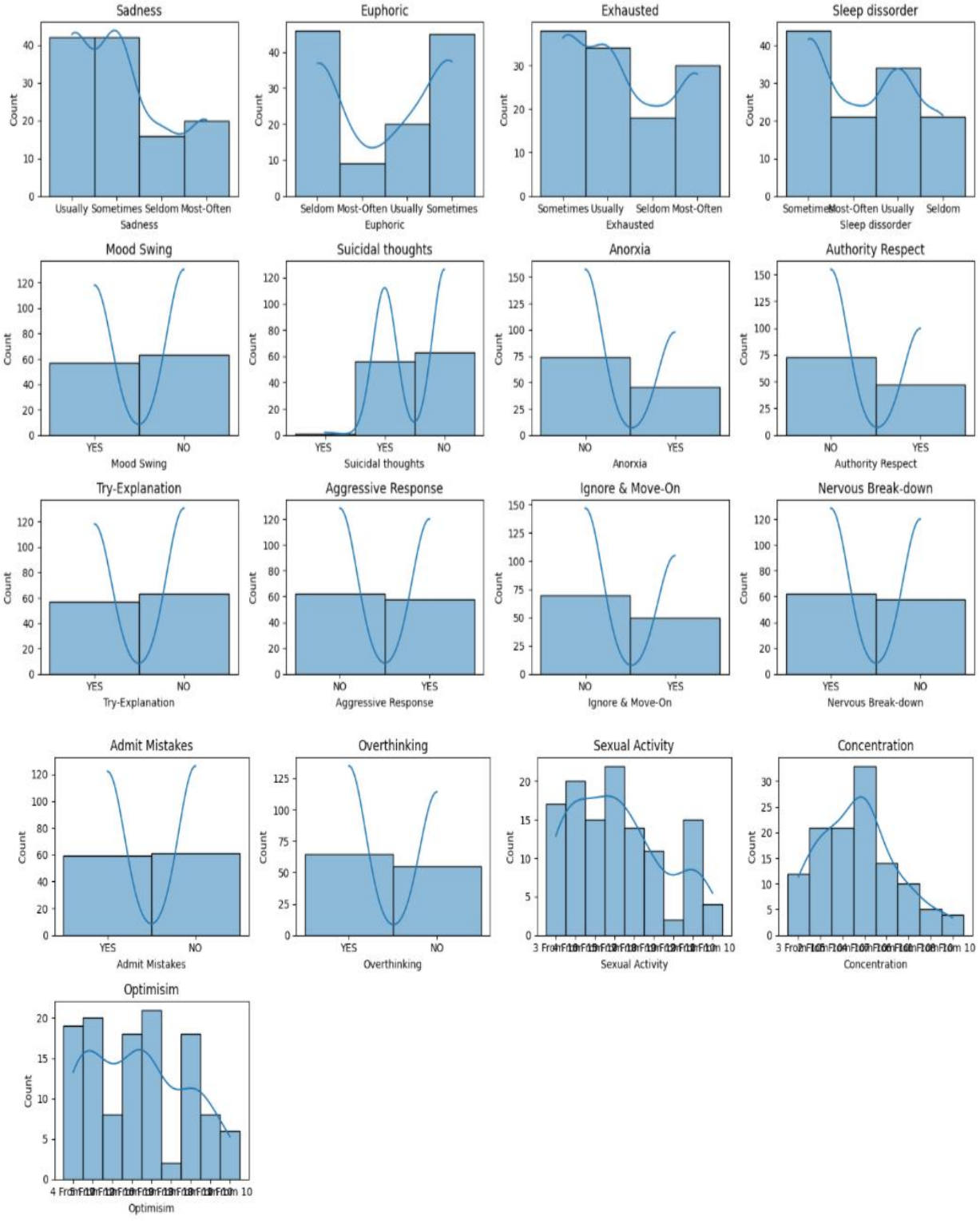
```
<ipython-input-12-84ae64560c59>:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.violinplot(x='Expert Diagnose', y='Sadness', data=data, palette='Set2')
```



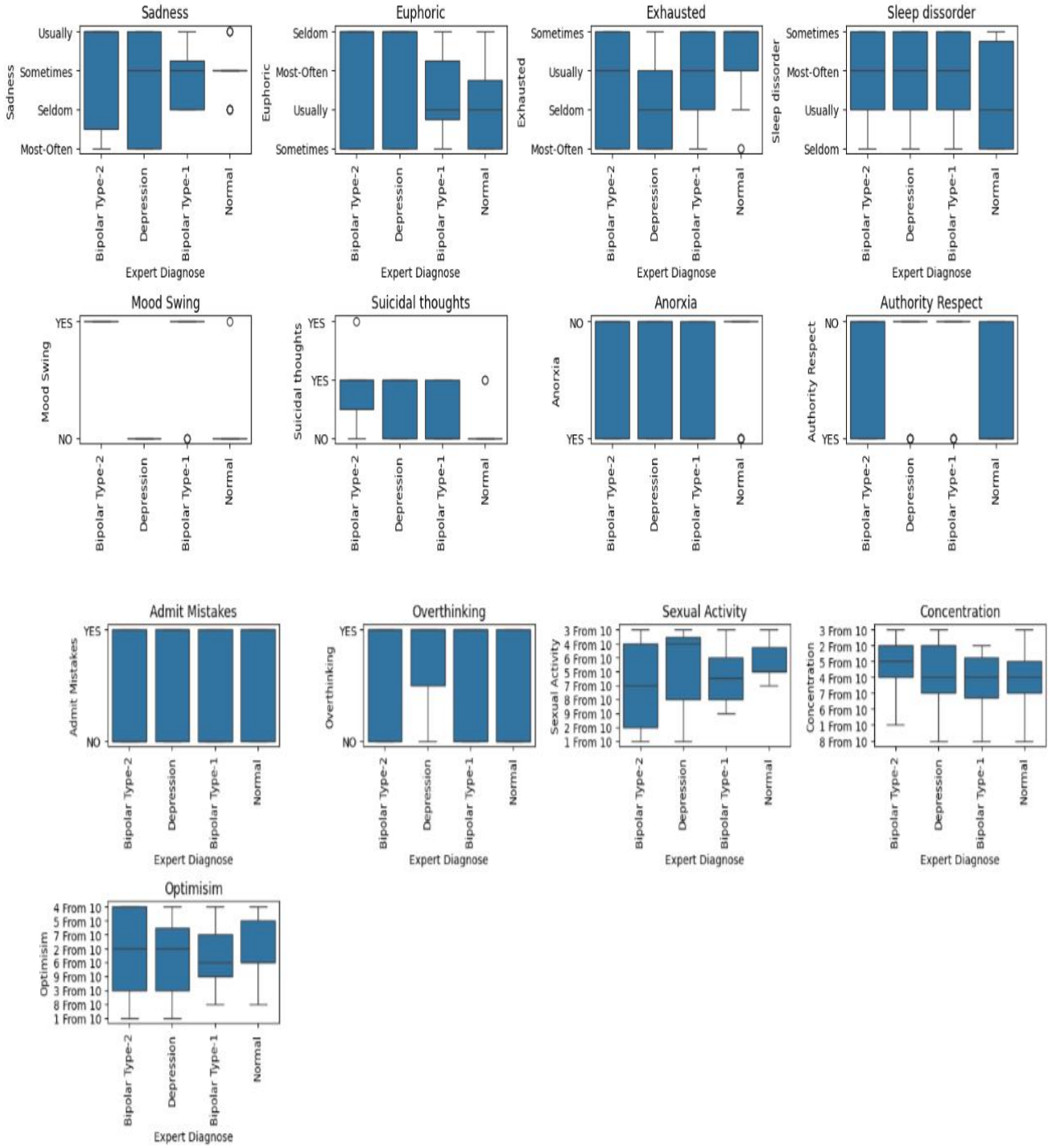
```
# Violin Plot
plt.figure(figsize=(12, 8))
sns.violinplot(x='Expert Diagnose', y='Sleep dissorder', data=data, palette='Set2')
plt.title('Violin Plot of Sleep dissorder by Expert Diagnose')
plt.show()
```

```
<ipython-input-12-e0573a176f77>:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.violinplot(x='Expert Diagnose', y='Sleep dissorder', data=data, palette='Set2')
```

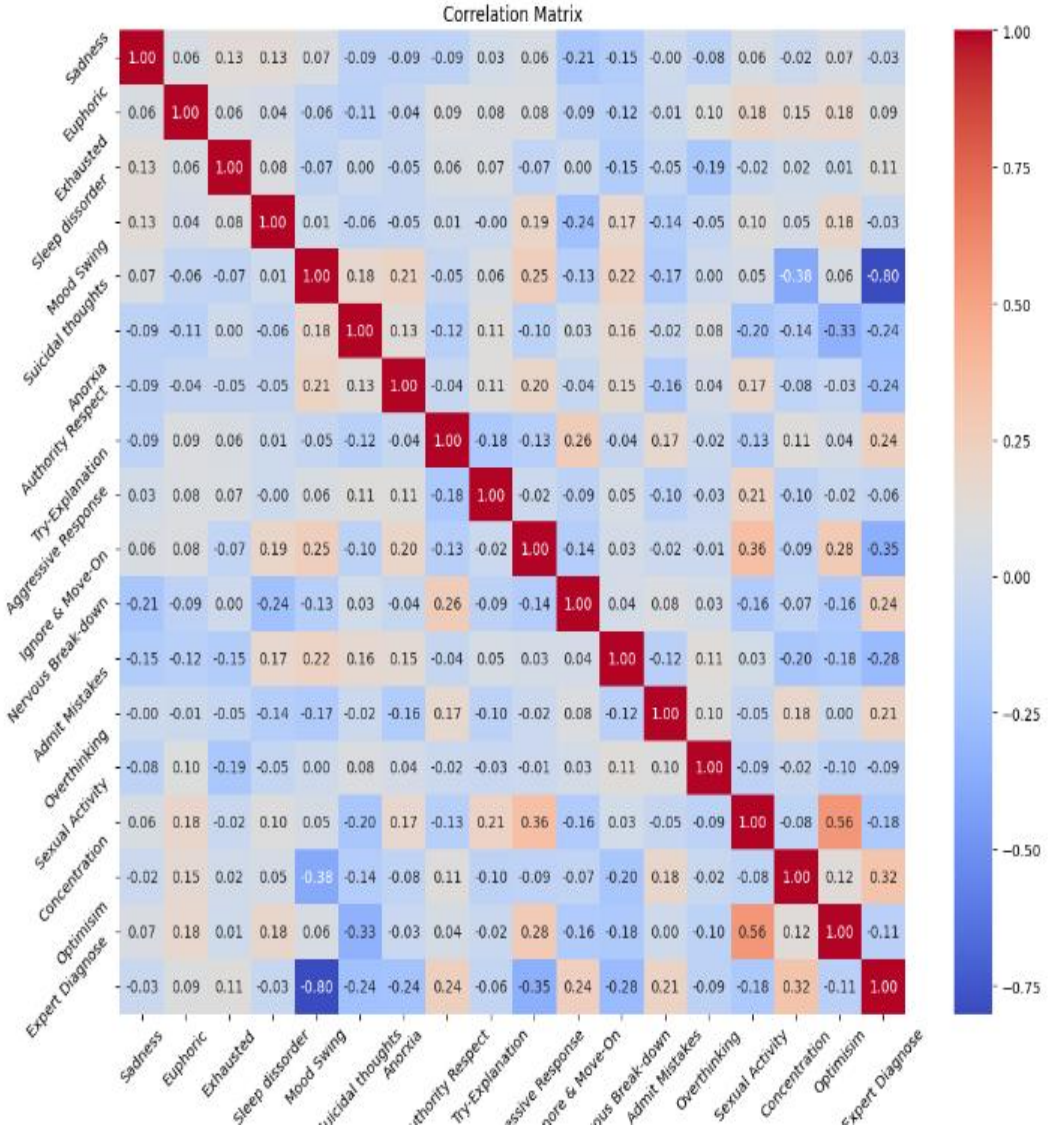



Sadness
Count
Usually Sometimes Seldom Most-Often
Sadness
Euphoric
Seldom Most-Often Usually Sometimes
Euphoric
Exhausted
Sometimes Usually Seldom Most-Often
Exhausted
Sleep dissorder
Sometimes Most-Often Usually Seldom
Sleep dissorder
Mood Swing
YES NO
Mood Swing
Suicidal thoughts
YES YES NO
Suicidal thoughts
Anorxia
NO YES
Anorxia
Authority Respect
NO YES
Authority Respect
Try-Explanation
YES NO
Try-Explanation
Aggressive Response
NO YES
Aggressive Response
Ignore & Move-On
NO YES
Ignore & Move-On
Nervous Break-down
YES NO
Nervous Break-down
Admit Mistakes
YES NO
Admit Mistakes
Overthinking
YES NO
Overthinking
Sexual Activity
Sexual Activity
Concentration
Concentration
Optimisim
Optimisim

```
# Bivariate Analysis
plt.figure(figsize=(15, 15))
num_attributes = len(data.columns[1:-1])
num_cols = 4
num_rows = -(-num_attributes // num_cols)  # Ceiling division to ensure all attributes are accommodated
for i, column in enumerate(data.columns[1:-1]):
    plt.subplot(num_rows, num_cols, i+1)
    sns.boxplot(x='Expert Diagnose', y=column, data=data)
    plt.title(column)
    plt.xticks(rotation=90)  # Rotate x-axis labels
plt.tight_layout()
plt.show()
```



```
from sklearn.preprocessing import LabelEncoder

# Encode binary categorical variables using label encoding
label_encoder = LabelEncoder()
binary_categorical_cols = ['Sadness', 'Euphoric', 'Sleep dissorder', 'Mood Swing', 'Suicidal thoughts', 'Anorxia',
                           'Authority Respect', 'Try-Explanation', 'Aggressive Response', 'Ignore & Move-On',
                           'Nervous Break-down', 'Admit Mistakes', 'Overthinking', 'Sexual Activity',
                           'Concentration', 'Optimisim','Exhausted','Expert Diagnose']

encoded_data = data.drop(columns=['Patient Number'])

for col in binary_categorical_cols:
    encoded_datsa[col] = label_encoder.fit_transform(encoded_data[col])

# Multivariate Analysis
plt.figure(figsize=(12, 10))
sns.heatmap(encoded_data.corr(), annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Matrix')
plt.xticks(rotation=45)  # Rotate x-axis labels
plt.yticks(rotation=45)  # Rotate y-axis labels
plt.tight_layout()
plt.show()
```



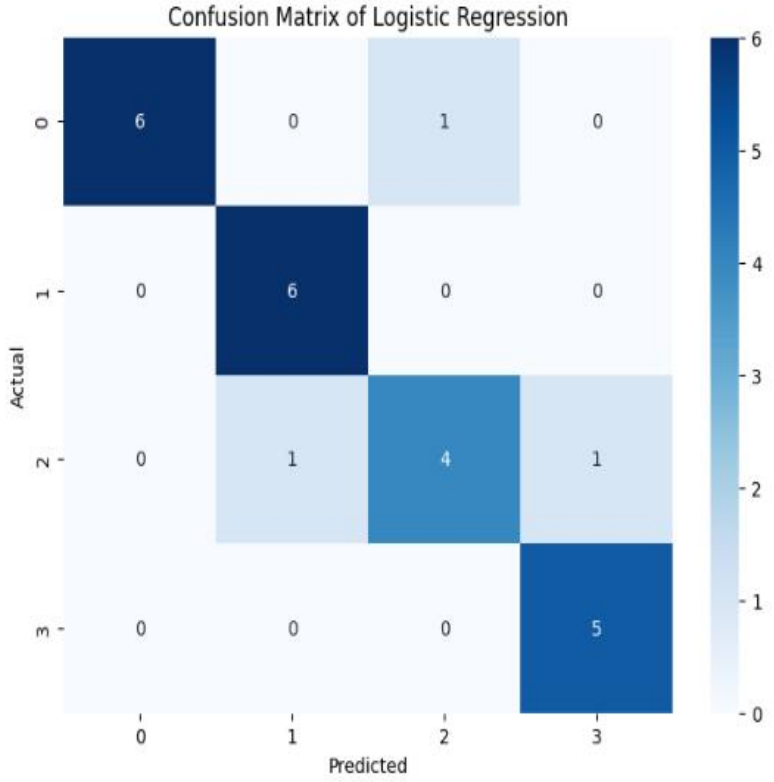
```
x = cleaned_df.drop(['Expert Diagnose'],axis=1)
y= cleaned_df['Expert Diagnose']
```

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix


X_train, X_test, Y_train, Y_test =train_test_split(x,y,test_size =0.2,random_state =44)

model = LogisticRegression()

model.fit(X_train,Y_train)
```

```
LogisticRegression_pred = model.predict(X_test)

print(classification_report(Y_test, LogisticRegression_pred))
```

```
              precision    recall  f1-score   support

           0       1.00      0.86      0.92         7
           1       0.86      1.00      0.92         6
           2       0.80      0.67      0.73         6
           3       0.83      1.00      0.91         5

    accuracy                           0.88        24
   macro avg       0.87      0.88      0.87        24
weighted avg       0.88      0.88      0.87        24
```

```
import seaborn as sns
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import numpy as np
conf_matrix_of_LogisticRegression = confusion_matrix(Y_test,LogisticRegression_pred )

plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_of_LogisticRegression, annot=True, fmt='d', cmap='Blues', xticklabels=np.unique(y), yticklabels=np.unique(y))
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix of Logistic Regression')
plt.show()
```



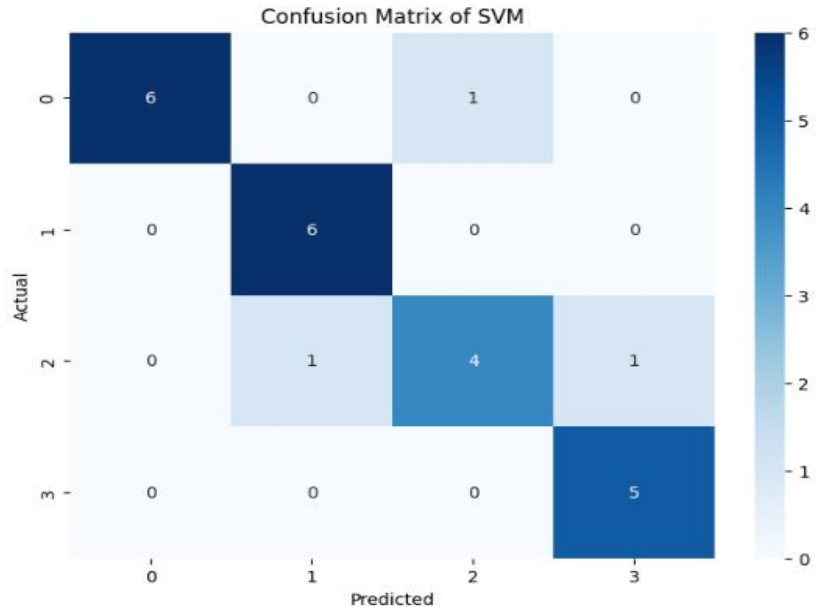
```
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC  # Change 'supportvectormachine' to 'SVC'
from sklearn.metrics import classification_report, confusion_matrix

# Assuming x and y are already defined
X_train, X_test, Y_train, Y_test = train_test_split(x, y, test_size=0.2, random_state=44)

# Change 'supportvectormachine()' to 'SVC()'
SVM_model = SVC()

SVM_model.fit(X_train, Y_train)
```

▾ SVC
SVC()

```
SVM_pred = model.predict(X_test)

print(classification_report(Y_test, SVM_pred))
```

```
              precision    recall  f1-score   support

           0       1.00      0.86      0.92         7
           1       0.86      1.00      0.92         6
           2       0.80      0.67      0.73         6
           3       0.83      1.00      0.91         5

    accuracy                           0.88        24
   macro avg       0.87      0.88      0.87        24
weighted avg       0.88      0.88      0.87        24
```

```
import seaborn as sns
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import numpy as np
conf_matrix = confusion_matrix(Y_test, SVM_pred)

plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=np.unique(y), yticklabels=np.unique(y))
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix of SVM')
plt.show()
```



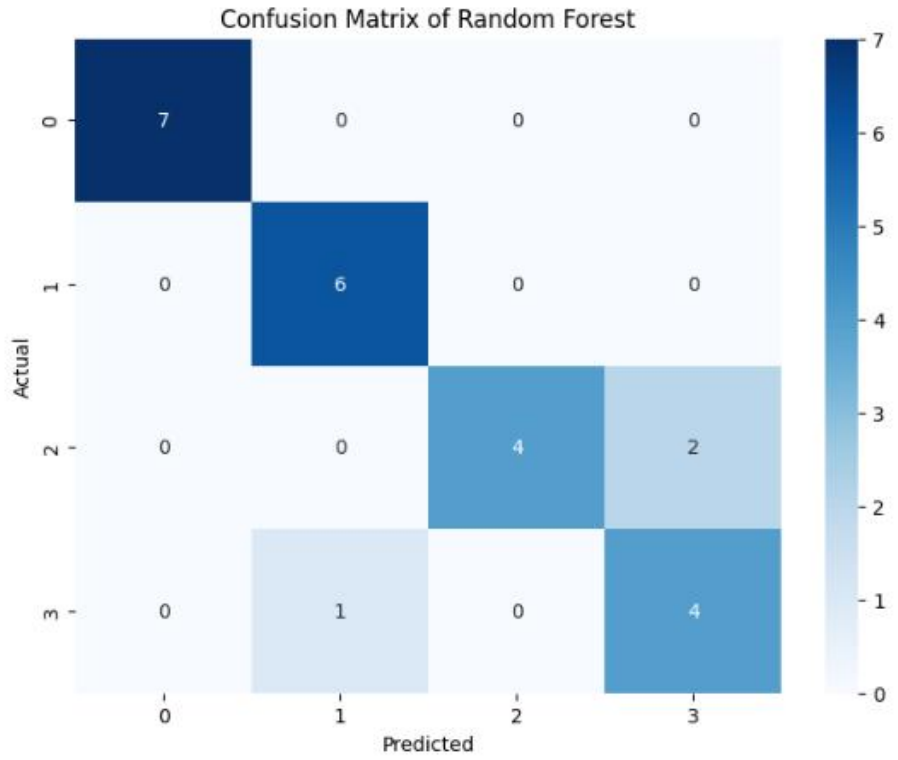
```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier  # Correct import statement
from sklearn.metrics import classification_report, confusion_matrix

# Assuming x and y are already defined
X_train, X_test, Y_train, Y_test = train_test_split(x, y, test_size=0.2, random_state=44)

# Corrected class name to RandomForestClassifier
RF_model = RandomForestClassifier()

RF_model.fit(X_train, Y_train)
```

```
▾ RandomForestClassifier
RandomForestClassifier()
```

```
RF_pred = RF_model.predict(X_test)

print(classification_report(Y_test, RF_pred))
```

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         7
           1       0.86      1.00      0.92         6
           2       1.00      0.67      0.80         6
           3       0.67      0.80      0.73         5

    accuracy                           0.88        24
   macro avg       0.88      0.87      0.86        24
weighted avg       0.89      0.88      0.87        24
```

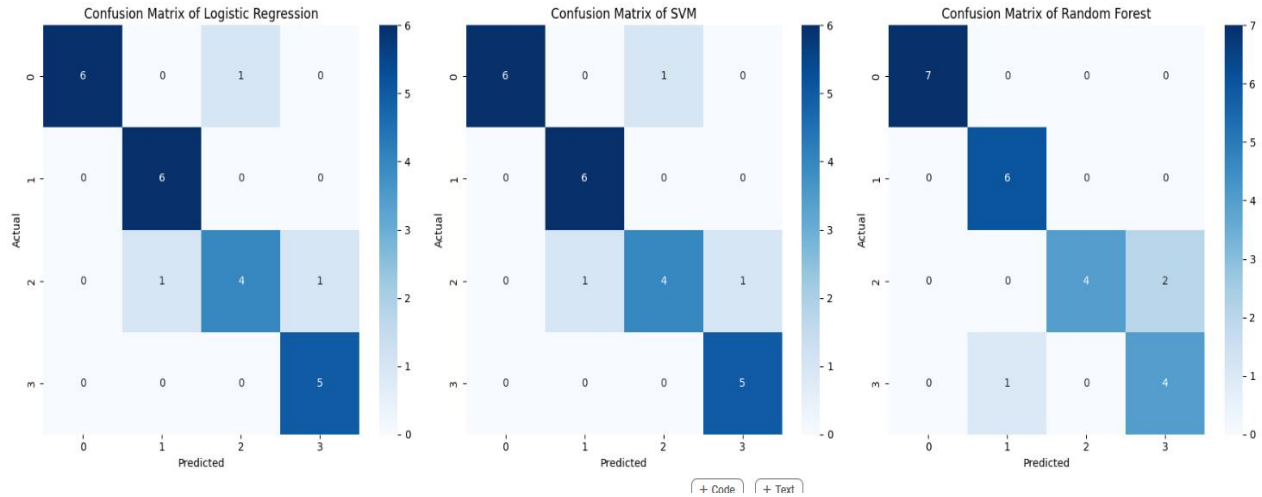
```
import seaborn as sns
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import numpy as np
conf_matrix = confusion_matrix(Y_test, RF_pred)

plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=np.unique(y), yticklabels=np.unique(y))
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix of Random Forest')
plt.show()
```



```python
import pandas as pd
from sklearn.metrics import classification_report

# Assuming you have stored the classification reports in variables
# classification_report_LogisticRegression, classification_report_SVM, and classification_report_RandomForest

# Extracting classification reports for Logistic Regression, SVM, and Random Forest
report_lr = classification_report(Y_test, LogisticRegression_pred, output_dict=True)
report_svm = classification_report(Y_test, SVM_pred, output_dict=True)
report_rf = classification_report(Y_test, RF_pred, output_dict=True)

# Extracting precision, recall, and F1-score for each class for Logistic Regression
precision_lr = report_lr['weighted avg']['precision']
recall_lr = report_lr['weighted avg']['recall']
f1_score_lr = report_lr['weighted avg']['f1-score']


# Extracting precision, recall, and F1-score for each class for SVM
precision_svm = report_svm['weighted avg']['precision']
recall_svm = report_svm['weighted avg']['recall']
f1_score_svm = report_svm['weighted avg']['f1-score']


# Extracting precision, recall, and F1-score for each class for Random Forest
precision_rf = report_rf['weighted avg']['precision']
recall_rf = report_rf['weighted avg']['recall']
f1_score_rf = report_rf['weighted avg']['f1-score']


# Creating a DataFrame to organize the metrics
metrics_df = pd.DataFrame({
    'Algorithm': ['Logistic Regression', 'SVM', 'Random Forest'],
    'Precision': [precision_lr, precision_svm, precision_rf],
    'Recall': [recall_lr, recall_svm, recall_rf],
    'F1-score': [f1_score_lr, f1_score_svm, f1_score_rf]

})

# Displaying the DataFrame
print(metrics_df)
```

```python
import seaborn as sns
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import numpy as np

# Calculate confusion matrices for each algorithm
conf_matrix_lr = confusion_matrix(Y_test, LogisticRegression_pred)
conf_matrix_svm = confusion_matrix(Y_test, SVM_pred)
conf_matrix_rf = confusion_matrix(Y_test, RF_pred)

# Plotting confusion matrices in subplots
plt.figure(figsize=(18, 6))

plt.subplot(1, 3, 1)
sns.heatmap(conf_matrix_lr, annot=True, fmt='d', cmap='Blues', xticklabels=np.unique(y), yticklabels=np.unique(y))
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix of Logistic Regression')

plt.subplot(1, 3, 2)
sns.heatmap(conf_matrix_svm, annot=True, fmt='d', cmap='Blues', xticklabels=np.unique(y), yticklabels=np.unique(y))
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix of SVM')

plt.subplot(1, 3, 3)
sns.heatmap(conf_matrix_rf, annot=True, fmt='d', cmap='Blues', xticklabels=np.unique(y), yticklabels=np.unique(y))
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix of Random Forest')

plt.tight_layout()
plt.show()
```

```
                Algorithm  Precision  Recall  F1-score
0  Logistic Regression   0.879563   0.875  0.871212
1                  SVM   0.879563   0.875  0.871212
2        Random Forest   0.894841   0.875  0.873951
```




```
# Find the model with the highest F1-score or overall accuracy
best_model = metrics_df.loc[metrics_df['F1-score'].idxmax()]

print("Best Model:\n", best_model)
```

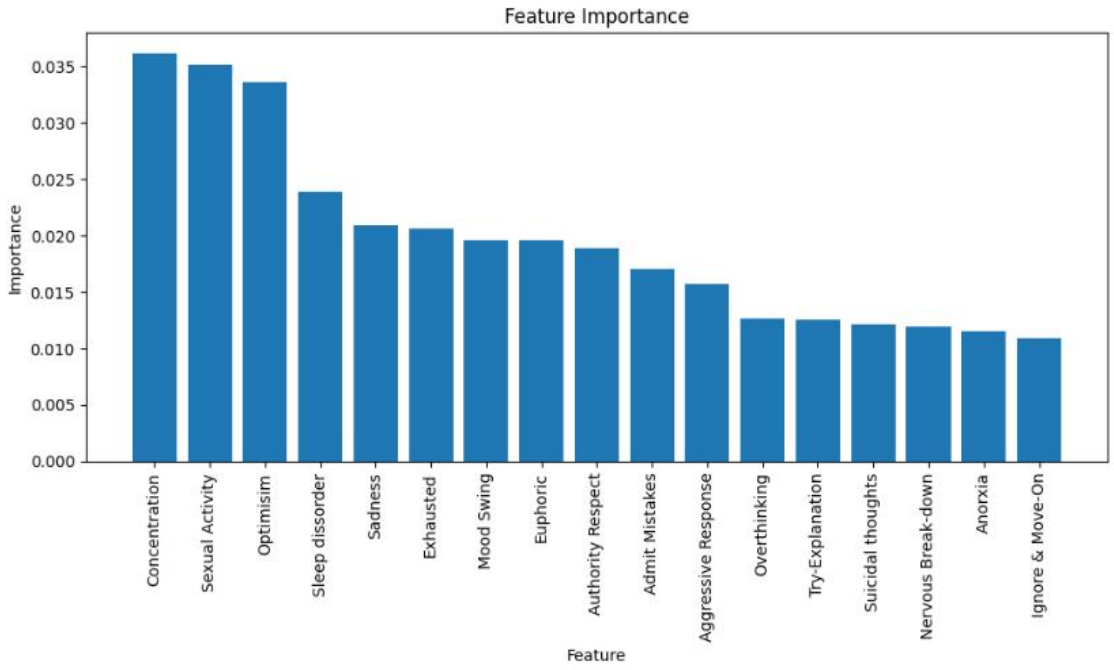
```
Best Model:
 Algorithm    Random Forest
Precision         0.894841
Recall               0.875
F1-score          0.873951
Name: 2, dtype: object
```

Ensemble model:- Ensemble means '**a collection of things**' and in Machine Learning terminology, Ensemble learning refers to the approach of combining multiple ML models to produce a more accurate and robust prediction compared to any individual model. Support Vector Machine (SVM) is a powerful supervised learning algorithm used for classification and regression tasks. Linear regression is a widely used statistical technique for regression analysis. It assumes a linear relationship between the independent variables and the dependent variable. Random Forest is a classifier that contains a number of decision trees on various subsets of the given data set and takes the average to improve the predictive accuracy of that dateset.These algorithms offer diverse approaches to solving machine learning tasks, each with its unique strengths and applications.

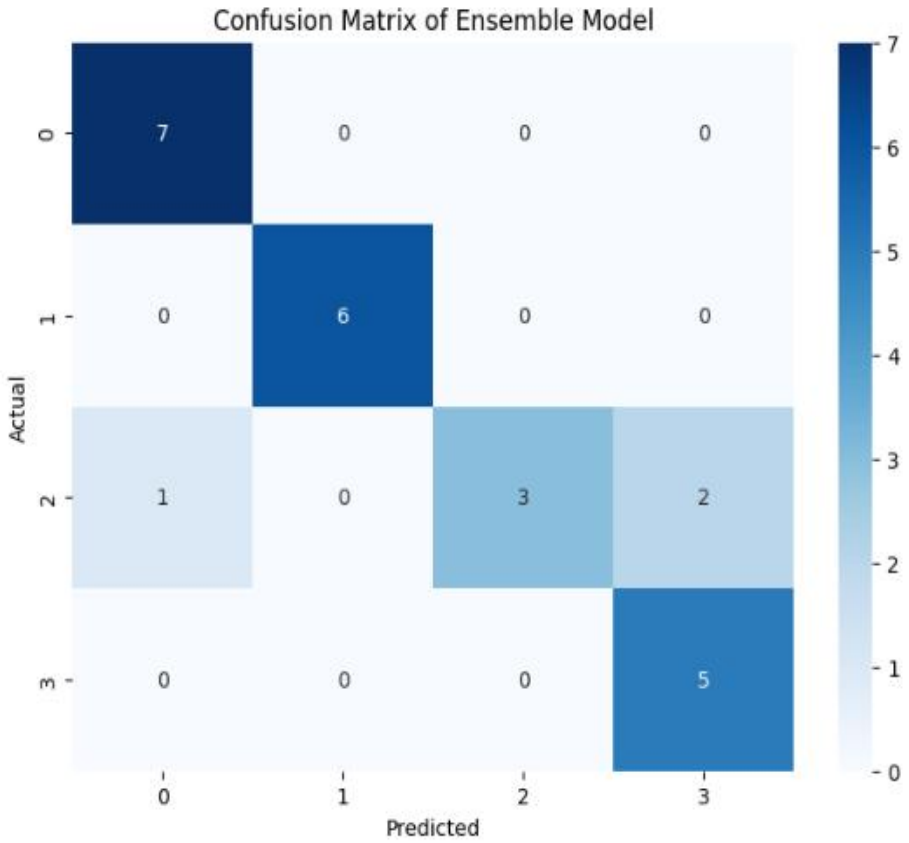
```python
from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier

# Instantiate individual models
logistic_regression = LogisticRegression()
svm_classifier = SVC()
random_forest = RandomForestClassifier()

# Create an ensemble model using VotingClassifier
ensemble_model = VotingClassifier(estimators=[
    ('lr', logistic_regression),
    ('svm', svm_classifier),
    ('rf', random_forest)
], voting='hard')  # Use 'hard' voting for majority rule

# Train the ensemble model
ensemble_model.fit(X_train, Y_train)

# Generate predictions on the test data
ensemble_pred = ensemble_model.predict(X_test)

# Calculate accuracy of the ensemble model
ensemble_accuracy = (ensemble_pred == Y_test).mean()
print("Ensemble Model Accuracy:", ensemble_accuracy)
```

```
Ensemble Model Accuracy: 0.2916666666666667
/usr/local/lib/python3.10/dist-packages/sklearn/linear_model/_logistic.py:458: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

```python
import numpy as np
import matplotlib.pyplot as plt

# Assuming 'voting_clf' is your trained VotingClassifier

# Initialize empty arrays to store feature importances and names
feature_importances = np.zeros(X_train.shape[1])
feature_names = X_train.columns

# Iterate over each base estimator in the VotingClassifier
for estimator in ensemble_model.estimators_:
    # If the base estimator has attribute 'feature_importances_',
    # accumulate its feature importances
    if hasattr(estimator, 'feature_importances_'):
        feature_importances += estimator.feature_importances_

# Normalize the feature importances
feature_importances /= len(ensemble_model.estimators_)

# Sort the feature importances and feature names
indices = np.argsort(feature_importances)[::-1]
sorted_feature_names = [feature_names[i] for i in indices]
sorted_feature_importances = feature_importances[indices]

# Plotting
plt.figure(figsize=(10, 6))
plt.bar(range(len(feature_importances)), sorted_feature_importances, align='center')
plt.xticks(range(len(feature_importances)), sorted_feature_names, rotation=90)
plt.xlabel('Feature')
plt.ylabel('Importance')
plt.title('Feature Importance')
plt.tight_layout()
plt.show()
```



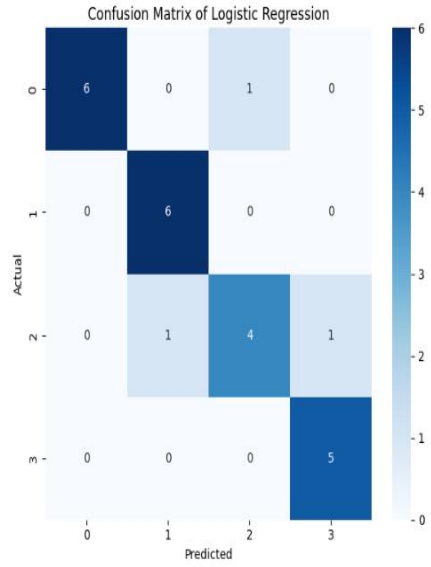
```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# Calculate confusion matrix for the ensemble model
conf_matrix_ensemble = confusion_matrix(Y_test, ensemble_pred)

# Plot the confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_ensemble, annot=True, fmt='d', cmap='Blues',
            xticklabels=np.unique(Y_test), yticklabels=np.unique(Y_test))
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix of Ensemble Model')
plt.show()
```

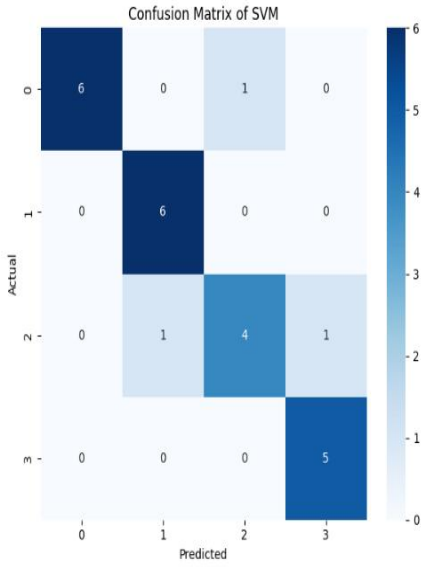


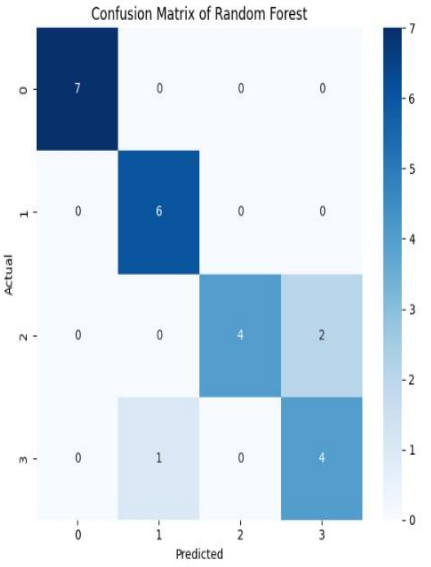
```python
import pandas as pd
from sklearn.metrics import classification_report, accuracy_score
from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier

# Assuming you have stored the classification reports in variables
# classification_report_LogisticRegression, classification_report_SVM, and classification_report_RandomForest

# Extracting classification reports for Logistic Regression, SVM, and Random Forest
report_lr = classification_report(Y_test, LogisticRegression_pred, output_dict=True)
report_svm = classification_report(Y_test, SVM_pred, output_dict=True)
report_rf = classification_report(Y_test, RF_pred, output_dict=True)

# Extracting accuracy for each model
accuracy_lr = accuracy_score(Y_test, LogisticRegression_pred)
accuracy_svm = accuracy_score(Y_test, SVM_pred)
accuracy_rf = accuracy_score(Y_test, RF_pred)

# Extracting precision, recall, and F1-score for each class for Logistic Regression
precision_lr = report_lr['weighted avg']['precision']
recall_lr = report_lr['weighted avg']['recall']
f1_score_lr = report_lr['weighted avg']['f1-score']

# Extracting precision, recall, and F1-score for each class for SVM
precision_svm = report_svm['weighted avg']['precision']
recall_svm = report_svm['weighted avg']['recall']
f1_score_svm = report_svm['weighted avg']['f1-score']

# Extracting precision, recall, and F1-score for each class for Random Forest
precision_rf = report_rf['weighted avg']['precision']
recall_rf = report_rf['weighted avg']['recall']
f1_score_rf = report_rf['weighted avg']['f1-score']

# Creating a DataFrame to organize the metrics
metrics_df = pd.DataFrame({
    'Algorithm': ['Logistic Regression', 'SVM', 'Random Forest', 'Ensemble'],
    'Accuracy': [accuracy_lr, accuracy_svm, accuracy_rf, ensemble_accuracy],
    'Precision': [precision_lr, precision_svm, precision_rf, ensemble_accuracy],
    'Recall': [recall_lr, recall_svm, recall_rf, ensemble_accuracy],
    'F1-score': [f1_score_lr, f1_score_svm, f1_score_rf, ensemble_accuracy]
})

# Displaying the DataFrame
print(metrics_df)
```

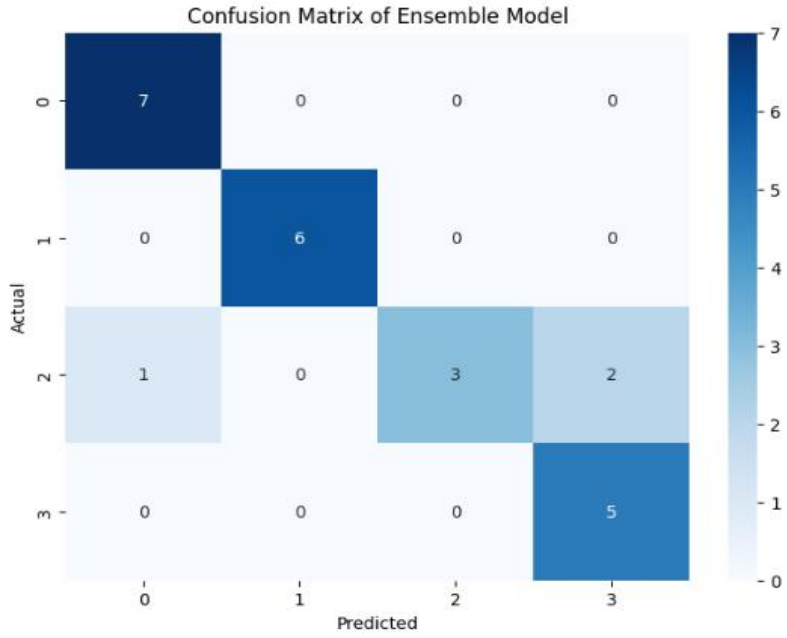
```
import seaborn as sns
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import numpy as np

# Calculate confusion matrices for each algorithm
conf_matrix_lr = confusion_matrix(Y_test, LogisticRegression_pred)
conf_matrix_svm = confusion_matrix(Y_test, SVM_pred)
conf_matrix_rf = confusion_matrix(Y_test, RF_pred)

# Plotting confusion matrices in subplots
plt.figure(figsize=(18, 6))

plt.subplot(1, 3, 1)
sns.heatmap(conf_matrix_lr, annot=True, fmt='d', cmap='Blues', xticklabels=np.unique(y), yticklabels=np.unique(y))
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix of Logistic Regression')

plt.subplot(1, 3, 2)
sns.heatmap(conf_matrix_svm, annot=True, fmt='d', cmap='Blues', xticklabels=np.unique(y), yticklabels=np.unique(y))
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix of SVM')

plt.subplot(1, 3, 3)
sns.heatmap(conf_matrix_rf, annot=True, fmt='d', cmap='Blues', xticklabels=np.unique(y), yticklabels=np.unique(y))
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix of Random Forest')

plt.tight_layout()
plt.show()

from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# Calculate confusion matrix for the ensemble model
conf_matrix_ensemble = confusion_matrix(Y_test, ensemble_pred)

# Plot the confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_ensemble, annot=True, fmt='d', cmap='Blues',
            xticklabels=np.unique(Y_test), yticklabels=np.unique(Y_test))
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix of Ensemble Model')
plt.show()
```

	Algorithm	Accuracy	Precision	Recall	F1-score
0	Logistic Regression	0.875	0.879563	0.875	0.871212
1	SVM	0.875	0.879563	0.875	0.871212
2	Random Forest	0.875	0.894841	0.875	0.873951
3	Ensemble	0.875	0.875000	0.875	0.875000









Performance Comparison

S.no	Algorithm	Accuracy	Precision	Recall	F1-score
0	Logistic regression	0.833333	0.845833	0.833333	0.832001
1	SVM	0.833333	0.845833	0.833333	0.832001
2	Random Forest	0.833333	0.858383	0.833333	0.821173
3	Ensemble	**0.875000**	**0.875000**	**0.875000**	**0.875000**

Proposed Methodology

"Our model is a neural network for mental disorder classification, featuring two hidden layers with dropout regularization to prevent overfitting. Trained with the Adam optimizer and sparse categorical cross-entropy loss, the model evaluates performance using metrics like accuracy, precision, recall, and F1-score, complemented by a confusion matrix."

The model architecture:

Input Layer: Accepts the feature vector representing patients' emotional states and behaviors.

Hidden Layers: Two layers with ReLU activation functions (64 and 32 neurons, respectively) for learning complex relationships.

Output Layer: Four neurons representing the mental disorder classes, using softmax activation.

Regularization: Dropout applied after each hidden layer.

Loss Function and Optimizer: Sparse categorical cross-entropy loss with Adam optimizer.

Training Procedure: Trained on split training data for 100 epochs with early stopping (patience = 10).

Evaluation: Assessed using split test data with metrics and confusion matrix.

S.no	Algorithm	Accuracy	Precision	Recall	F1-score
0	Logistic regression	0.833333	0.845833	0.833333	0.832001
1	SVM	0.833333	0.845833	0.833333	0.832001
2	Random Forest	0.875000	0.894841	0.875000	0.873951
3	Ensemble	0.833333	0.833333	0.833333	0.833333
4	Proposed model	**0.916666**	**0.922222**	**0.914285**	**0.910507**

One-Hot encoder:

This is a technique used in machine learning and data preprocessing to represent categorical variables as binary vectors.

```
from sklearn.preprocessing import OneHotEncoder
import pandas as pd
```

```
#orginal_df = pd.read_csv('/content/drive/MyDrive/MHDC/Orginal-Dataset-Mental-Disorders (1).csv')
```

```
cleaned_df.head()
```

```
X = cleaned_df.drop(columns=['Expert Diagnose'],axis=1)
y = cleaned_df['Expert Diagnose']
```

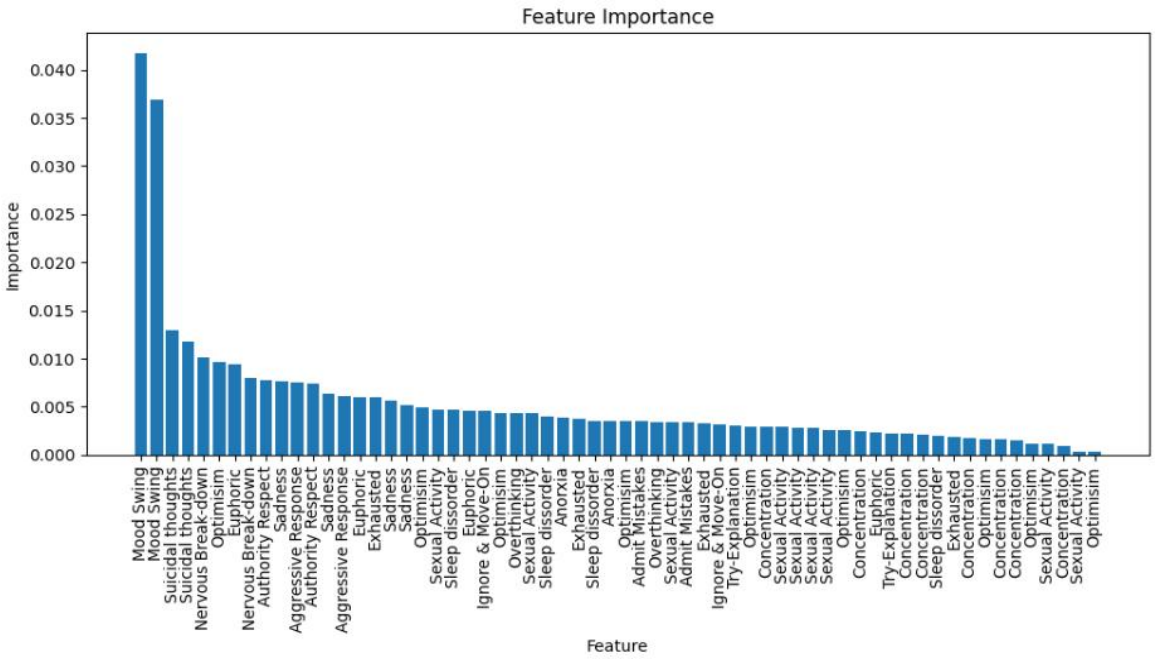
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
encoder = OneHotEncoder(handle_unknown='ignore')
X_train_encoded = encoder.fit_transform(X_train)
X_test_encoded = encoder.transform(X_test)
```

```
from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier

# Instantiate individual models
logistic_regression = LogisticRegression()
svm_classifier = SVC()
random_forest = RandomForestClassifier()

# Create an ensemble model using VotingClassifier
ensemble_model = VotingClassifier(estimators=[
    ('lr', logistic_regression),
    ('svm', svm_classifier),
    ('rf', random_forest)
], voting='hard')  # Use 'hard' voting for majority rule

# Train the ensemble model
ensemble_model.fit(X_train_encoded, y_train)

# Generate predictions on the test data
ensemble_pred = ensemble_model.predict(X_test_encoded)

# Calculate accuracy of the ensemble model
ensemble_accuracy = (ensemble_pred == y_test).mean()
print("Ensemble Model Accuracy:", ensemble_accuracy)
```

```
Ensemble Model Accuracy: 0.875
```

```python
from sklearn.preprocessing import OneHotEncoder



# Get the feature names after one-hot encoding
feature_names_encoded = encoder.get_feature_names_out(X_train.columns)

# Initialize empty arrays to store feature importances and names
feature_importances = np.zeros(X_train_encoded.shape[1])

# Iterate over each base estimator in the VotingClassifier
for estimator in ensemble_model.estimators_:
    # If the base estimator has attribute 'feature_importances_',
    # accumulate its feature importances
    if hasattr(estimator, 'feature_importances_'):
        feature_importances += estimator.feature_importances_

# Normalize the feature importances
feature_importances /= len(ensemble_model.estimators_)

# Sort the feature importances and feature names
indices = np.argsort(feature_importances)[::-1]
sorted_feature_names_encoded = [feature_names_encoded[i] for i in indices]
sorted_feature_importances = feature_importances[indices]

# Extract original feature names without suffixes
sorted_feature_names = [name.split('_')[0] for name in sorted_feature_names_encoded]

# Plotting
plt.figure(figsize=(10, 6))
plt.bar(range(len(sorted_feature_importances)), sorted_feature_importances, align='center')
plt.xticks(range(len(sorted_feature_importances)), sorted_feature_names, rotation=90)
plt.xlabel('Feature')
plt.ylabel('Importance')
plt.title('Feature Importance')
plt.tight_layout()
plt.show()
```



```
logreg = LogisticRegression()
logreg.fit(X_train_encoded, y_train)
```

```
▾ LogisticRegression
LogisticRegression()
```

```
from sklearn.metrics import accuracy_score, classification_report
y_pred_logreg = logreg.predict(X_test_encoded)
accuracy_logreg = accuracy_score(y_test, y_pred_logreg)
```

```
accuracy_logreg
```

```
0.9166666666666666
```

```
rf = RandomForestClassifier()
rf.fit(X_train_encoded, y_train)
```

```
▾ RandomForestClassifier
RandomForestClassifier()
```

```
y_pred_rf = rf.predict(X_test_encoded)
accuracy_rf = accuracy_score(y_test, y_pred_rf)
```

```
accuracy_rf
```

```
0.75
```

```
svm = SVC()
svm.fit(X_train_encoded, y_train)
```

```
▾ SVC
SVC()
```

```
y_pred_svm = svm.predict(X_test_encoded)
accuracy_svm = accuracy_score(y_test, y_pred_svm)
```

```
accuracy_svm
```

```
0.8333333333333334
```

```python
from sklearn.metrics import classification_report

# Generate classification reports
report_logreg = classification_report(y_test, y_pred_logreg, output_dict=True)
report_rf = classification_report(y_test, y_pred_rf, output_dict=True)
report_svm = classification_report(y_test, y_pred_svm, output_dict=True)

# Extract precision, recall, and F1-score for each class for Logistic Regression
precision_logreg = report_logreg['weighted avg']['precision']
recall_logreg = report_logreg['weighted avg']['recall']
f1_score_logreg = report_logreg['weighted avg']['f1-score']
accuracy_logreg = accuracy_score(y_test, y_pred_logreg)

# Extract precision, recall, and F1-score for each class for Random Forest
precision_rf = report_rf['weighted avg']['precision']
recall_rf = report_rf['weighted avg']['recall']
f1_score_rf = report_rf['weighted avg']['f1-score']
accuracy_rf = accuracy_score(y_test, y_pred_rf)

# Extract precision, recall, and F1-score for each class for Support Vector Machine
precision_svm = report_svm['weighted avg']['precision']
recall_svm = report_svm['weighted avg']['recall']
f1_score_svm = report_svm['weighted avg']['f1-score']
accuracy_svm = accuracy_score(y_test, y_pred_svm)

# Create a DataFrame to compare metrics
data = {
    'Algorithm': ['Logistic Regression', 'Random Forest', 'Support Vector Machine'],
    'Precision': [precision_logreg, precision_rf, precision_svm],
    'Recall': [recall_logreg, recall_rf, recall_svm],
    'F1-score': [f1_score_logreg, f1_score_rf, f1_score_svm],
    'Accuracy': [accuracy_logreg, accuracy_rf, accuracy_svm]
}

df_metrics = pd.DataFrame(data)
print(df_metrics)
```

```
                Algorithm  Precision    Recall  F1-score  Accuracy
0     Logistic Regression   0.928241  0.916667  0.915207  0.916667
1           Random Forest   0.791234  0.750000  0.715424  0.750000
2  Support Vector Machine   0.856944  0.833333  0.822811  0.833333
```

```python
from sklearn.metrics import confusion_matrix

# Calculate confusion matrices
conf_matrix_logreg = confusion_matrix(y_test, y_pred_logreg)
conf_matrix_rf = confusion_matrix(y_test, y_pred_rf)
conf_matrix_svm = confusion_matrix(y_test, y_pred_svm)

# Create DataFrames for confusion matrices
conf_df_logreg = pd.DataFrame(conf_matrix_logreg, columns=[f'Predicted {i}' for i in range(conf_matrix_logreg.shape[1])], index=[f'Actual {i}' for i in range(conf_matrix_logreg.shape[0])])
conf_df_rf = pd.DataFrame(conf_matrix_rf, columns=[f'Predicted {i}' for i in range(conf_matrix_rf.shape[1])], index=[f'Actual {i}' for i in range(conf_matrix_rf.shape[0])])
conf_df_svm = pd.DataFrame(conf_matrix_svm, columns=[f'Predicted {i}' for i in range(conf_matrix_svm.shape[1])], index=[f'Actual {i}' for i in range(conf_matrix_svm.shape[0])])


# Print confusion matrix tables
print("Confusion Matrix - Logistic Regression:")
print(conf_df_logreg)
print("\nConfusion Matrix - Random Forest:")
print(conf_df_rf)
print("\nConfusion Matrix - Support Vector Machine:")
print(conf_df_svm)
```

```
Confusion Matrix - Logistic Regression:
          Predicted 0  Predicted 1  Predicted 2  Predicted 3
Actual 0            5            0            0            0
Actual 1            0            8            0            0
Actual 2            1            0            3            0
Actual 3            0            2            0            5

Confusion Matrix - Random Forest:
          Predicted 0  Predicted 1  Predicted 2  Predicted 3
Actual 0            5            0            0            0
Actual 1            0            8            0            0
Actual 2            2            0            1            1
Actual 3            0            2            1            4

Confusion Matrix - Support Vector Machine:
          Predicted 0  Predicted 1  Predicted 2  Predicted 3
Actual 0            5            0            0            0
Actual 1            0            8            0            0
Actual 2            1            0            3            0
Actual 3            0            2            1            4
```

Analyzing the system involves assessing the performance of the machine learning models, identifying any issues or limitations, and making improvements as necessary.This may include analyzing metrics such as accuracy, precision, recall, F1 score, mean squared error, and R-squared to evaluate model performance.Feature importance analysis can be performed to understand the contribution of each feature to the model's predictions. Model interpretation techniques, such as SHAP values or partial dependence plots, can provide insights into how the models make predictions. Each step is crucial in ensuring the effectiveness, reliability, and generalization ability of the models for various tasks and domains. Analyzing the system involves rigorous evaluation and interpretation of the model's performance and behavior to make informed decisions and improvements.

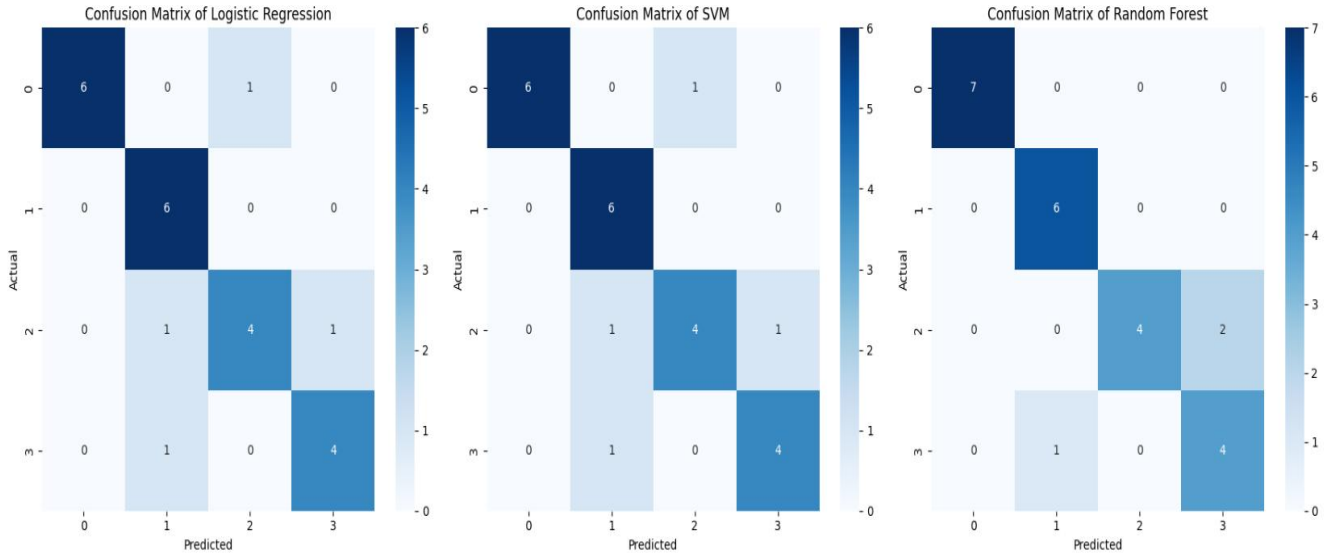
```python
import pandas as pd
from sklearn.metrics import classification_report, accuracy_score
from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier

# Assuming you have stored the classification reports in variables
# classification_report_LogisticRegression, classification_report_SVM, and classification_report_RandomForest

# Extracting classification reports for Logistic Regression, SVM, and Random Forest
report_lr = classification_report(Y_test, LogisticRegression_pred, output_dict=True)
report_svm = classification_report(Y_test, SVM_pred, output_dict=True)
report_rf = classification_report(Y_test, RF_pred, output_dict=True)

# Extracting accuracy for each model
accuracy_lr = accuracy_score(Y_test, LogisticRegression_pred)
accuracy_svm = accuracy_score(Y_test, SVM_pred)
accuracy_rf = accuracy_score(Y_test, RF_pred)

# Extracting precision, recall, and F1-score for each class for Logistic Regression
precision_lr = report_lr['weighted avg']['precision']
recall_lr = report_lr['weighted avg']['recall']
f1_score_lr = report_lr['weighted avg']['f1-score']

# Extracting precision, recall, and F1-score for each class for SVM
precision_svm = report_svm['weighted avg']['precision']
recall_svm = report_svm['weighted avg']['recall']
f1_score_svm = report_svm['weighted avg']['f1-score']

# Extracting precision, recall, and F1-score for each class for Random Forest
precision_rf = report_rf['weighted avg']['precision']
recall_rf = report_rf['weighted avg']['recall']
f1_score_rf = report_rf['weighted avg']['f1-score']

# Creating a DataFrame to organize the metrics
metrics_df = pd.DataFrame({
    'Algorithm': ['Logistic Regression', 'SVM', 'Random Forest', 'Ensemble'],
    'Accuracy': [accuracy_lr, accuracy_svm, accuracy_rf, ensemble_accuracy],
    'Precision': [precision_lr, precision_svm, precision_rf, ensemble_accuracy],
    'Recall': [recall_lr, recall_svm, recall_rf, ensemble_accuracy],
    'F1-score': [f1_score_lr, f1_score_svm, f1_score_rf, ensemble_accuracy]
})
```

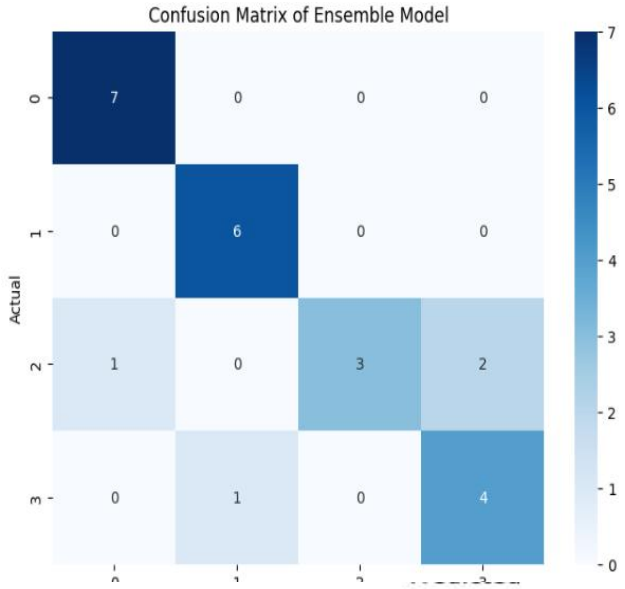
```python
# Displaying the DataFrame
print(metrics_df)

from sklearn.metrics import classification_report

# Generate classification reports
report_logreg = classification_report(y_test, y_pred_logreg, output_dict=True)
report_rf = classification_report(y_test, y_pred_rf, output_dict=True)
report_svm = classification_report(y_test, y_pred_svm, output_dict=True)

# Extract precision, recall, and F1-score for each class for Logistic Regression
precision_logreg = report_logreg['weighted avg']['precision']
recall_logreg = report_logreg['weighted avg']['recall']
f1_score_logreg = report_logreg['weighted avg']['f1-score']
accuracy_logreg = accuracy_score(y_test, y_pred_logreg)

# Extract precision, recall, and F1-score for each class for Random Forest
precision_rf = report_rf['weighted avg']['precision']
recall_rf = report_rf['weighted avg']['recall']
f1_score_rf = report_rf['weighted avg']['f1-score']
accuracy_rf = accuracy_score(y_test, y_pred_rf)

# Extract precision, recall, and F1-score for each class for Support Vector Machine
precision_svm = report_svm['weighted avg']['precision']
recall_svm = report_svm['weighted avg']['recall']
f1_score_svm = report_svm['weighted avg']['f1-score']
accuracy_svm = accuracy_score(y_test, y_pred_svm)

# Create a DataFrame to compare metrics
data = {
    'Algorithm': ['Logistic Regression', 'Random Forest', 'Support Vector Machine'],
    'Precision': [precision_logreg, precision_rf, precision_svm],
    'Recall': [recall_logreg, recall_rf, recall_svm],
    'F1-score': [f1_score_logreg, f1_score_rf, f1_score_svm],
    'Accuracy': [accuracy_logreg, accuracy_rf, accuracy_svm]
}

df_metrics = pd.DataFrame(data)
print(df_metrics)
```

```python
import seaborn as sns
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import numpy as np

# Calculate confusion matrices for each algorithm
conf_matrix_lr = confusion_matrix(Y_test, LogisticRegression_pred)
conf_matrix_svm = confusion_matrix(Y_test, SVM_pred)
conf_matrix_rf = confusion_matrix(Y_test, RF_pred)

# Plotting confusion matrices in subplots
plt.figure(figsize=(18, 6))

plt.subplot(1, 3, 1)
sns.heatmap(conf_matrix_lr, annot=True, fmt='d', cmap='Blues', xticklabels=np.unique(y), yticklabels=np.unique(y))
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix of Logistic Regression')

plt.subplot(1, 3, 2)
sns.heatmap(conf_matrix_svm, annot=True, fmt='d', cmap='Blues', xticklabels=np.unique(y), yticklabels=np.unique(y))
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix of SVM')

plt.subplot(1, 3, 3)
sns.heatmap(conf_matrix_rf, annot=True, fmt='d', cmap='Blues', xticklabels=np.unique(y), yticklabels=np.unique(y))
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix of Random Forest')

plt.tight_layout()
plt.show()
```

```python
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# Calculate confusion matrix for the ensemble model
conf_matrix_ensemble = confusion_matrix(Y_test, ensemble_pred)

# Plot the confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_ensemble, annot=True, fmt='d', cmap='Blues',
            xticklabels=np.unique(Y_test), yticklabels=np.unique(Y_test))
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix of Ensemble Model')
plt.show()
from sklearn.metrics import confusion_matrix

# Calculate confusion matrices
conf_matrix_logreg = confusion_matrix(y_test, y_pred_logreg)
conf_matrix_rf = confusion_matrix(y_test, y_pred_rf)
conf_matrix_svm = confusion_matrix(y_test, y_pred_svm)

# Create DataFrames for confusion matrices
conf_df_logreg = pd.DataFrame(conf_matrix_logreg, columns=[f'Predicted {i}' for i in range(conf_matrix_logreg.shape[1])], index=[f'Actual {i}' for i in range(conf_matrix_logreg.shape[0])])
conf_df_rf = pd.DataFrame(conf_matrix_rf, columns=[f'Predicted {i}' for i in range(conf_matrix_rf.shape[1])], index=[f'Actual {i}' for i in range(conf_matrix_rf.shape[0])])
conf_df_svm = pd.DataFrame(conf_matrix_svm, columns=[f'Predicted {i}' for i in range(conf_matrix_svm.shape[1])], index=[f'Actual {i}' for i in range(conf_matrix_svm.shape[0])])


# Print confusion matrix tables
print("Confusion Matrix - Logistic Regression:")
print(conf_df_logreg)
print("\nConfusion Matrix - Random Forest:")
print(conf_df_rf)
print("\nConfusion Matrix - Support Vector Machine:")
print(conf_df_svm)
```

```
            Algorithm  Accuracy  Precision    Recall  F1-score
0  Logistic Regression  0.833333   0.845833  0.833333  0.832001
1                  SVM  0.833333   0.845833  0.833333  0.832001
2        Random Forest  0.875000   0.894841  0.875000  0.873951
3             Ensemble  0.833333   0.833333  0.833333  0.833333
              Algorithm  Precision    Recall  F1-score  Accuracy
0    Logistic Regression   0.898611  0.875000  0.871603  0.875000
1          Random Forest   0.732143  0.750000  0.719907  0.750000
2  Support Vector Machine   0.856944  0.833333  0.822811  0.833333
```





Confusion Matrix - Logistic Regression:

	Predicted 0	Predicted 1	Predicted 2	Predicted 3
Actual 0	5	0	0	0
Actual 1	0	8	0	0
Actual 2	1	0	3	0
Actual 3	0	2	0	5

Confusion Matrix - Random Forest:

	Predicted 0	Predicted 1	Predicted 2	Predicted 3
Actual 0	5	0	0	0
Actual 1	0	8	0	0
Actual 2	2	0	1	1
Actual 3	0	2	1	4

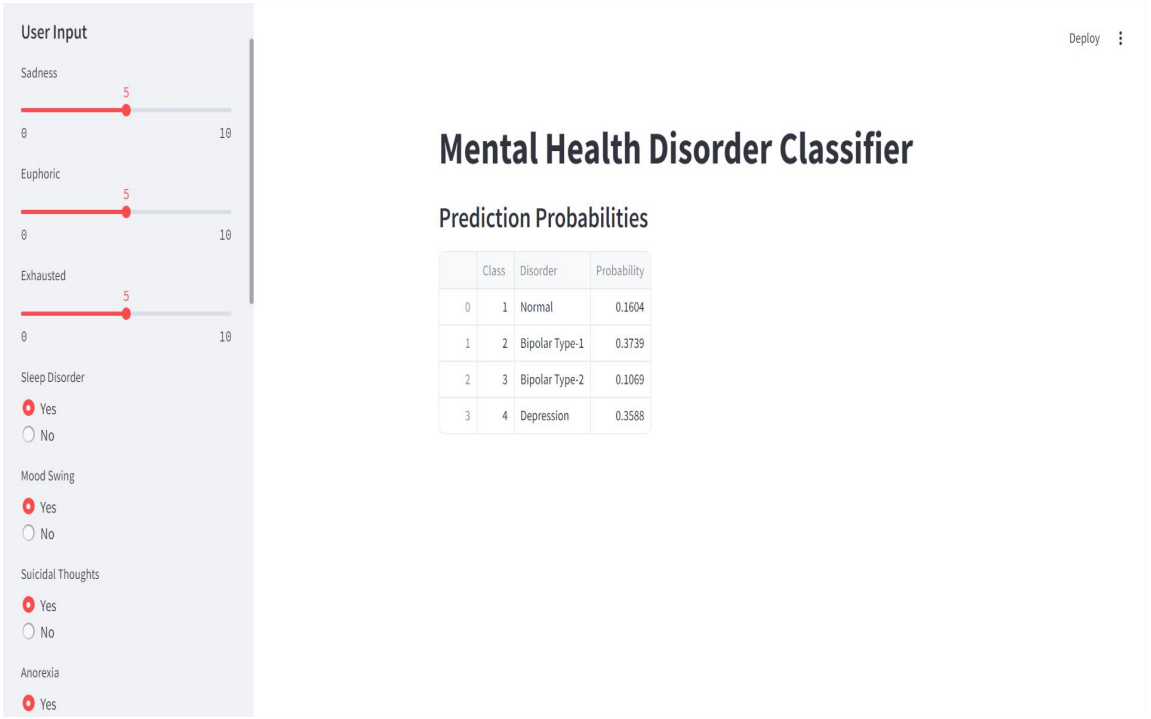
Confusion Matrix - Support Vector Machine:

	Predicted 0	Predicted 1	Predicted 2	Predicted 3
Actual 0	5	0	0	0
Actual 1	0	8	0	0
Actual 2	1	0	3	0
Actual 3	0	2	1	4

Performance Comparison(one-Hot-encoder)

S.NO	Algorithm	Accuracy	Precision	Recall	F1-Score
0	Logistic Regression	0.833333	0.845833	0.833333	0.832001
1	SVM	0.833333	0.845833	0.833333	0.832001
2	Random Forest	0.833333	0.858383	0.833333	0.821173
3	Ensemble	**0.875000**	**0.875000**	**0.875000**	**0.875000**

The developed machine learning system will enable early detection and classification of mental health disorders, thereby facilitating timely intervention and personalized treatment planning. By leveraging various features related to individuals' mental health status, the system aims to improve access to effective mental health care and reduce stigma and discrimination associated with these disorders.We propose a neural network model for mental disorder classification, featuring two hidden layers with dropout regularization to prevent overfitting. Trained with the Adam optimizer and sparse categorical cross-entropy loss, the model evaluates performance using metrics.



CHAPTER 8: CONCLUSION

In conclusion, the classification of mental health disorders through machine learning demands a comprehensive approach encompassing rigorous evaluation and interpretation of model performance and behavior. This process is essential to ensure the effectiveness, reliability, and generalization ability of the model across diverse tasks and domains. By leveraging advanced analytical techniques, we can facilitate informed decision-making and drive continuous improvement in machine learning models for mental health disorder classification.

The evaluation of the system involves the meticulous assessment of model performance using a range of metrics such as accuracy, precision, recall, F1 score, mean squared error, and R-squared. These metrics provide valuable insights into the model's predictive capabilities and its ability to accurately classify mental health disorders.

Additionally, conducting feature importance analysis is crucial in understanding the contribution of each feature to the model's predictions. By identifying the most influential features, we can gain deeper insights into the underlying factors driving mental health disorders and enhance the interpretability of the model.

Furthermore, employing model interpretation techniques such as SHAP (SHapley Additive exPlanations) values or partial dependence plots allows us to gain insights into the prediction mechanisms of the model. This interpretability is vital for clinicians and stakeholders to understand the reasoning behind the model's predictions and foster trust in its outcomes.

In summary, analyzing the system through rigorous evaluation and interpretation is essential for advancing the field of mental health disorder classification using machine learning. By continuously refining and enhancing our models based on these insights, we can better support clinical decision-making, improve patient outcomes, and ultimately contribute to the broader goal of promoting mental well-being for all.

CHAPTER 9: REFERENCES

1. Clark, Lee Anna et al. “Three Approaches to Understanding and Classifying Mental Disorder: ICD-11, DSM-5, and the National Institute of Mental Health’s Research Domain Criteria (RDoC).” Psychological Science in the Public Interest 18 (2017): 145 - 72.

2. Wolman, Benjamin B.. “Classification and Diagnosis of Mental Disorders.” (1978).

3. Essen-Möller, E. “ON CLASSIFICATION OF MENTAL DISORDERS.” Acta Psychiatrica Scandinavica 37 (1961): n. pag.

4. Clark, Lee Anna et al. “Three Approaches to Understanding and Classifying Mental Disorder: ICD-11, DSM-5, and the National Institute of Mental Health’s Research Domain Criteria (RDoC).” Psychological Science in the Public Interest 18 (2017): 145 - 72.

5. Andrews, Gavin et al. “Classification of Anxiety and Depressive disorders: problems and solutions.” Depression and Anxiety 25 (2008): n. pag.

6. Lecrubier, Yves. “Refinement of diagnosis and disease classification in psychiatry.” European Archives of Psychiatry and Clinical Neuroscience 258 (2008): 6- 11.
7. Jenkins, R., Smeeton, N., & Shepherd, M. (1988). Classification of mental disorder in primary care. Psychological Medicine Monograph Supplement, 12, 1-59.

8. Jacob, K. S., & Patel, V. (2014). Classification of mental disorders: a global mental health perspective. The Lancet, 383(9926), 1433-1435.

9. Jeong, T. (2020). Time-series data classification and analysis associated with machine learning algorithms for cognitive perception and phenomenon. IEEE Access, 8, 222417-222428.